



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



INSTITUTO TECNOLÓGICO DE MORELIA
"José María Morelos y Pavón"

INSTITUTO TECNOLÓGICO DE MORELIA

DIVISIÓN DE ESTUDIOS PROFESIONALES

DEPARTAMENTO DE METAL MECÁNICA

Ingeniería Mecatrónica

Reporte de Residencias Profesionales

**“DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA DE
COMUNICACIÓN Y MONITOREO ENTRE PLC INDUSTRIAL Y
PLATAFORMA DE ANÁLISIS DE DATOS UTILIZANDO
RASPBERRY PI”**

PRESENTA:

IAN LUIS BÁRCENAS MORENO

ASESORES:

**CARLOS ALBERTO GUIZAR GÓMEZ
PEDRO LÓPEZ MARTÍNEZ**

MORELIA, MICHOACÁN

31 de Julio de 2026

Agradecimientos

Expreso mi inmensa gratitud a las personas que han brindado su apoyo y confianza a la ejecución del proyecto. Es el caso del ingeniero **Pedro López Martínez** de la empresa Mitchell Plastics Querétaro. Como su asesorado, considero un privilegio ser su subordinado, y no puedo evitar mencionar que es sumamente grata la nobleza y claridad con la que dirige al equipo de automatización en la planta.

También agradezco a mis compañeros de trabajo en el área de automatización; **Kevin Salgado, Diego Martínez, Arturo Neri, Juan Manuel (El Inge), Adán** por su apoyo al brindarme conocimientos prácticos muy valiosos y por hacer muy amena mi estancia en la empresa y en la ciudad de Querétaro.

Mi familia, concretamente mi madre, **Laura del Carmen Moreno Frías**, y mi padre, **Juan Luis Bárcenas Flores**, quienes merecen toda la admiración y agradecimiento de mi parte por ser en este proyecto, así como en el resto de mi vida, un constante recordatorio de que no debo rendirme jamás y un pilar para mi motivación en desarrollar este proyecto y mejorar mis habilidades como profesional.

Resumen

En este informe se recopilan los métodos, resultados y tecnologías que se implementaron en un sistema de adquisición, monitoreo y visualización de datos en una estación de ensamble. En Mitchell Plastics Querétaro, nombre comercial de la empresa canadiense Ultra Manufacturing S.A. de C.V., se fabrican componentes de plástico para interiores de automóviles.

En este caso particular, se trabajó con una estación de ensamblado final de manijas para las partes interiores de las puertas de una camioneta de toyota como parte del departamento de automatización de la empresa. El proyecto constó de las siguientes etapas:

1. Se configura una **Raspberry Pi 5** para contar con los siguientes elementos:
 - **PostgreSQL** para manejo de una base de datos.
 - **MQTT** para comunicación entre PLC Omron N1XP2 y Raspberry Pi 5.
 - **Node-RED** para desarrollar programación dirigida por eventos de alto nivel.
 - **Grafana** para monitorización de datos por medio de paneles de control (dashboards) en tiempo real.
2. Se establecieron los parámetros que se registrarán en la base de datos, en este caso, se trabajó con tres variables categóricas, una numérica, y dos de tipo fecha que corresponden a diferentes variables del proceso en cuestión que se revisarán más a detalle en la sección de desarrollo.
3. En PostgreSQL se creó la base de datos en la que se estarán registrando y consultando los datos recabados en la estación.
4. En Node-RED, por otra parte, se desarrolló un flujo que, dados ciertos eventos (como recibir un dato desde MQTT, o que llegué el cambio de turno) se ejecuten diferentes scripts como enviar un email de alertas de calidad y de lentitud automáticamente.
5. Paralelamente al flujo de Node-RED, en Grafana se crearon dos paneles de control; Uno en el que se muestran las piezas ensambladas cada hora en cada turno (matutino y vespertino), y otro en que se muestran tendencias de calidad.

Este proyecto es pionero en la implementación de la industria 4.0 en Mitchell Plastics Querétaro. Por medio de este se busca cubrir los 4 pilares de la industria 4.0 en la estación; **Visibilidad, Transparencia, Predicción y Autonomía.**

Índice general

1. Generalidades del Proyecto	9
1.1. Introducción	9
1.2. Descripción de la Empresa y Puesto	10
1.3. Problemas a Resolver	11
1.3.1. Costo económico excesivo	11
1.3.2. Registros semiautomáticos y error humano	11
1.3.3. Trazabilidad deficiente	11
1.4. Objetivos	12
1.4.1. Objetivo General	12
1.4.2. Objetivos Específicos	12
1.5. Justificación	12
2. Marco Teórico	14
2.1. Antecedentes	14
2.2. Elementos del Proceso	15
2.2.1. Cámara IV4 Keyence	16
2.2.2. Robot FANUC	16
2.2.3. Cámara Serie VS y Prueba Eléctrica	17
2.3. Secuencia de la Estación	18
2.4. PLC Omron NX1P2	20
2.5. Industria 4.0	21
2.5.1. IoT con Raspberry Pi 5	22
2.6. Protocolos de Comunicación	23
2.6.1. Ethernet/IP	23
2.6.2. MQTT	25
2.6.3. SMTP	26
2.6.4. JSON y Diccionarios	26
2.7. Software Utilizado	28
2.7.1. Debian Raspberry Pi Linux	28
2.7.2. Sysmac Studio	30

2.7.3.	Node-RED	30
2.7.4.	JavaScript	32
2.7.5.	HTML	34
2.7.6.	PostgreSQL	35
2.7.7.	Grafana	38
3.	Desarrollo	41
3.1.	Configuración Inicial de la Raspberry Pi 5	42
3.2.	Instalación de Programas	43
3.2.1.	Mosquitto Broker y Node-RED	43
3.2.2.	Instalación de PostgreSQL	44
3.2.3.	Instalación de Grafana	45
3.2.4.	pgAdmin 4	47
3.3.	Variables del Proceso y Comunicación con MQTT	47
3.4.	Procesamiento de los Registros en Node-RED	50
3.5.	Creación de Base de Datos	50
3.6.	Programación de Reportes de Producción por Turno en Node-RED	51
3.6.1.	Formato de Email	52
3.7.	Desarrollo de Alertas del Sistema	53
3.8.	Desarrollo de Tablero en Grafana	56
4.	Resultados	57
4.1.	Programas Realizados	57
4.2.	Base de Datos	58
4.3.	Tableros en Grafana.	58
4.4.	Emails Automáticos	59
5.	Conclusiones	62
6.	Competencias Desarrolladas	64

Índice de figuras

1.1. Logo y Ubicación de Mitchell Plastics Querétaro. Obtenido de https://www.mitchellplastics.com/es/locations	10
2.1. Manija ensamblada en la estación 400D.	16
2.2. Cámaras IV4 extraída de la página web de Keyence F. A. Corporation, 2016	16
2.3. Ejemplo de robot LR Mate 200 iD	17
2.4. Elementos de inspección en la estación de ensamble 400D.	18
2.5. Nido donde se colocan las manijas sobre la mesa indexadora.	19
2.6. PLC Omron NX1P2	21
2.7. Raspberry Pi 5 y sus componentes.	23
2.8. Conexiones por Ethernet/IP en el proyecto.	24
2.9. Cables con conectores RJ45 para conexiones con PLC.	25
2.10. Comunicaciones en el proyecto utilizando JSON y hash tables.	28
2.11. Interfaz gráfica de Sysmac Studio	30
2.12. Flujo de ejemplo de Node-RED	31
2.13. Interfaz Gráfica de pgAdmin 4	37
2.14. Tablero de ejemplo en Grafana	39
2.15. Panel de ejemplo en Grafana	40
3.1. Diagrama de flujo de la secuencia de la estación de ensamble.	41
3.2. Interfaz de Raspberry Pi Imager.	42
3.3. Cambios en el archivo de configuración pg_hba.conf	45
3.4. Archivo de configuración para acceso remoto. Autoría propia, desarrollado en Mitchell Plastics Querétaro.	46
3.5. Bloques de función para establecer conexión con el broker mosquitto, publicar, enviar ping de conexión, y suscribirse a tópicos	49
3.6. Construcción del JSON en Sysmac Studio (La obtención de las variables no corresponde a este proyecto y el código para procesarlas es de uso exclusivo para Mitchell Plastics)	49
3.7. Flujo y contenido de los nodos en Node-RED.	50
3.8. Base de datos en pgAdmin 4	51
3.9. Flujo de envío de reportes de producción por turno en Node-RED.	52
3.10. Código para generar el formato del Email.	52

3.11. Construcción del JSON para enviar señales de alerta.	53
3.12. Flujo de manejo de alertas.	54
4.1. Diagramas de escalera para alertas del proceso.	57
4.2. Flujos desarrollados en Node-RED.	57
4.3. Base de datos operativa.	58
4.4. [Pantalla en la estación 400D con tableros]Pantalla en la estación 400D con tableros. Autoría propia, desarrollado en Mitchell Plastics Querétaro.	59
4.5. [Pantalla en la estación 400D con tableros]Tablero con tabla de producción hora por hora. Autoría propia, desarrollado en Mitchell Plastics Querétaro.	59
4.6. Email reporte de turno.	60
4.7. Email de alerta de calidad.	60
4.8. Email de alerta de ciclos lentos.	61

Índice de tablas

2.1. Modelos que se ensamblan en la estación 400D	19
2.2. Niveles de la industria 4.0	22
2.3. Directorios de utilidad de Linux.	29
2.4. Nodos que se usarán para desarrollar el proyecto. Gran parte de la información se obtuvo de la página oficial de Node-RED O’Leary y Conway-Jones, 2026.	32
3.1. Ejemplo de registro que se llevará a la base de datos.	48

Lista de Códigos

2.1. JSON cuyos valores varían en formato.	27
2.2. Ejemplo de Object en JavaScript extraído de Svekis et al., 2021	33
2.3. Accesos a variables dentro de un Object extraído de Svekis et al., 2021	33
2.4. Estructura de un documento HTML. Elaboracion propia label	34
2.5. Uso de Inline CSS. Elaboracion propia	35
2.6. Creación de una tabla con SQL. Elaboracion propia	37
3.1. Comandos para gestionar la red ethernet sin Network Manager. Autoria propia, desarrollado en Mitchell Plastics Queretaro	42
3.2. Comandos para instalar Node-RED y Mosquitto Broker. Autoria propia, desarrollado en Mitchell Plastics Queretaro.	43
3.3. Comandos para habilitar conexiones desde otros dispositivos para MQTT, y crear un primer usuario. Autoria propia, desarrollado en Mitchell Plastics Queretaro.	44
3.4. Comandos para instalar PostgreSQL, y crear un primer usuario. Autoria propia, desarrollado en Mitchell Plastics Queretaro.	44
3.5. Comandos para editar el archivo de configuración de PostgreSQL. Autoria propia, desarrollado en Mitchell Plastics Queretaro.	44
3.6. Comando para editar el archivo de configuración de PostgreSQL. Autoria propia, desarrollado en Mitchell Plastics Queretaro.	45
3.7. Comandos para instalar y generar los archivos necesarios para Grafana. Autoria propia, desarrollado en Mitchell Plastics Queretaro.	46
3.8. Creación de archivos e instalación de pgadmin4 en la terminal de linux. Autoría propia, desarrollado en Mitchell Plastics Querétaro.	47
3.9. Configuración de nodemailer para enviar correos electrónicos, Autoría propia	55

Capítulo 1

Generalidades del Proyecto

1.1. Introducción

La transformación digital de la industria manufacturera ha impulsado la incorporación de tecnologías orientadas a la automatización, el monitoreo de procesos y la gestión inteligente de la información. Conceptos como Industria 4.0 e Internet de las Cosas permiten la interconexión de dispositivos de control, sensores y sistemas de supervisión, facilitando la adquisición de datos en tiempo real para mejorar la eficiencia operativa, la trazabilidad y la toma de decisiones dentro de los procesos productivos.

En la industria automotriz, particularmente en la fabricación de autopartes, donde los estándares de calidad y confiabilidad son cada vez más exigentes, la disponibilidad de información de proceso constituye un elemento esencial para identificar fallas, reducir tiempos de respuesta y optimizar el desempeño de las líneas de producción. Hoy existen muchas tecnologías para conectar dispositivos entre ellos, recopilar información, y crear algoritmos de respuesta a diferentes necesidades de monitoreo en los procesos industriales.

Mitchell Plastics Querétaro, empresa dedicada a la fabricación de componentes plásticos para la industria automotriz, ha incorporado diversas soluciones de automatización e IoT en sus procesos de manufactura. Sin embargo, dichas soluciones presentan limitaciones relacionadas con el almacenamiento de información, la automatización de registros y la integración entre los sistemas de monitoreo y las bases de datos, lo que ocasiona dependencia de hardware adicional, procesos semiautomáticos y una trazabilidad limitada de la información generada durante la producción.

Ante esta situación, el presente proyecto propone el diseño e implementación de un sistema de comunicación y monitoreo basado en una Raspberry Pi 5 integrada con un PLC Omron. La solución desarrollada permite establecer una arquitectura de adquisición de datos utilizando el protocolo MQTT para enviarlos al dispositivo de IoT, incorporar mecanismos para el almacenamiento automático de información en una base de datos relacional y proporcionar herramientas para la visualización del estado del proceso mediante paneles de monitoreo en tiempo real. Asimismo, el sistema incorpora programación orientada a eventos que permite automatizar diversas tareas sin intervenir en la lógica principal del control industrial.

El desarrollo de este proyecto busca demostrar la viabilidad de utilizar plataformas de bajo costo para implementar soluciones de monitoreo industrial con funcionalidades avanzadas de comunicación, almacenamiento y visualización de datos, contribuyendo a mejorar la trazabilidad de los procesos y sentando las bases para futuras implementaciones

de sistemas IIoT dentro de la empresa.

1.2. Descripción de la Empresa y Puesto

Ultra Manufacturing S.A. de C.V. es una empresa especializada en fabricación de autopartes con inyección de plástico y ha estado desarrollando proyectos para BMW, Stellantis, Jeep, Ford, Toyota, etc. desde 1997. La instalación en Querétaro es una de siete fábricas en Canadá, Estados Unidos y México. Tiene presencia en México desde 2014 y ha estado implementando importantes tecnologías como moldeo por inyección de plástico, soldaduras, cámaras inteligentes etc. En la Figura 1.1 aparecen el logo y la planta de la empresa donde se desarrolló el proyecto.



(a) Logo Oficial de Mitchell Plastics



(b) Planta en Querétaro

Figura 1.1: Logo y Ubicación de Mitchell Plastics Querétaro. Obtenido de <https://www.mitchellplastics.com/es/locations>

La empresa está seccionada en varias áreas. Producción, Mantenimiento, Procesos, Proyectos, RH, Calidad, IT y Automatización. Este proyecto fue desarrollado en el departamento de automatización en donde desempeño el cargo de practicante de automatización. En este puesto las principales tareas que se desarrollan son de programación esencialmente para los PLC's con los que se cuenta en la empresa; en su mayoría de la marca japonesa omron haciendo uso del software Sysmac Studio. El departamento trabaja en las integraciones mecánicas, electroneumáticas y eléctricas de las estaciones en la fábrica en coordinación con el resto de departamentos.

Dentro del puesto, además del proyecto como tal, se apoya en las integraciones descritas. Particularmente ha sido necesario programar PLC, sensores, cámaras inteligentes y HMI de la marca Keyence y robots Fanuc. Por añadidura, se brinda apoyo en la puesta en marcha de producción de las estaciones atendiendo fallas y haciendo ajuste fino para las revisiones de calidad.

1.3. Problemas a Resolver

En Mitchell Plastics Querétaro, si bien se han implementado sistemas de IoT en planta, estos se caracterizan por ser meramente pasivos en el sentido de que no ejecutan ninguna acción real sobre el proceso, ni siquiera guardan la información en bases de datos. Se pueden, entonces, enumerar los problemas que de esta situación surgen.

1.3.1. Costo económico excesivo

Los sistemas de monitoreo y IoT de la empresa seguían dos tendencias: O bien, se utiliza un PLC Omron de un modelo NJ bastante caro que puede gestionar una base de datos en el gestor de base de datos MySQL, o se usa una Raspberry Pi 5 y se conecta mediante MQTT al PLC para desplegar la información en paneles de control, pero el sistema carece de funciones para la base de datos, obligando al departamento a usar laptops colocadas en las estaciones para hacer registros en la base de datos mediante escáneres.

En ambos casos se incurre en costos adicionales que se podrían evitar si se aprovecharan mejor las capacidades de IoT de la Raspberry Pi 5 o cualquier otro dispositivo con el que se pueda crear un sistema que integre **Comunicación por MQTT, Programación por eventos, Gestión de base de datos, Automatización de tareas, y Monitorización de datos** en un sólo elemento de la estación.

1.3.2. Registros semiautomáticos y error humano

Como se mencionó en el punto anterior, los operadores de las estaciones tienen que registrar cada pieza que pasa por la estación de manera manual en la base de datos. Ello hace el proceso propenso a errores humanos y requiere hardware externo. Se plantea automatizar aún más el proceso para hacer registros automáticos según ciertos criterios en el proceso.

1.3.3. Trazabilidad deficiente

En Mitchell Plastics Querétaro han ocurrido graves fallas tanto en la producción como en el manejo de las piezas que se venden a los clientes. Se ha estado experimentando con soluciones de IoT para establecer trazabilidad en los procesos productivos de la planta buscando recopilar información de los mismos. No obstante, antes de este proyecto, esas soluciones en realidad no habían sido totalmente validadas y los datos realmente no llegaban a ninguna autoridad puesto que no se reportaban.

En otras palabras, se asumía que dichas soluciones funcionaban, pero, en realidad, los empleados seguían ciegos ante la información que esos sistemas de IoT recopilaban.

1.4. Objetivos

1.4.1. Objetivo General

Diseñar e implementar un sistema de comunicación y monitoreo entre un PLC industrial y una plataforma de análisis de datos utilizando Raspberry Pi, con el propósito de adquirir y visualizar información de proceso en una línea de ensamble.

1.4.2. Objetivos Específicos

1. Analizar la arquitectura básica de comunicación entre PLC, dispositivos industriales y plataforma de adquisición de datos.
2. Definir las variables de proceso que serán monitoreadas y registradas durante la prueba piloto.
3. Desarrollar la interfaz de comunicación entre el PLC omron y la Raspberry Pi 5 para la adquisición de datos.
4. Desarrollar un panel de control de monitoreo para la visualización de información en tiempo real.
5. Validar el funcionamiento del sistema en una aplicación piloto.
6. Documentar la arquitectura, funcionamiento y resultados obtenidos para su posible escalamiento futuro.

1.5. Justificación

En los entornos actuales de manufactura, la disponibilidad de información de proceso es fundamental para mejorar la trazabilidad, facilitar el análisis de fallas y apoyar la toma de decisiones técnicas.

Sin embargo, en muchas estaciones automatizadas, la información generada por el PLC y otros dispositivos no siempre se encuentra disponible de manera estructurada para su consulta, análisis o visualización.

A partir de esta necesidad, surge la propuesta de desarrollar un sistema de comunicación y monitoreo basado en Raspberry Pi como plataforma de adquisición y procesamiento de datos, integrada con un PLC Omron.

Esta solución busca aprovechar tecnologías accesibles y flexibles para generar una arquitectura funcional que permita capturar información relevante del proceso sin afectar la operación principal del sistema de control.

Capítulo 2

Marco Teórico

2.1. Antecedentes

Este proyecto emplea tecnologías, que si bien, ya se han implementado en la fábrica, no se ha explotado todo el potencial de los dispositivos empleados. Se han llevado a cabo proyectos con Raspberry Pi 5 y Arduino Opta los cuales se caracterizan por tener un diseño pasivo, en el que el dispositivo es incapaz de ejecutar decisiones. Esto es verdad para Mitchell Plastics Querétaro, pero también para muchas otras empresas dentro de la industria. Según la información interna de Mitchell Plastics tanto en México como en Canadá, las aplicaciones de IoT e Industria 4.0 que se han desarrollado han sido muy limitadas por dos razones:

1. La industria 4.0 requiere la integración de múltiples disciplinas y habilidades especializadas como microcontroladores, sistemas operativos o lenguajes de programación especializados. La empresa no ha contratado especialistas en este área, lo que ha limitado el desarrollo de soluciones innovadoras que aprovechen los dispositivos con los que la planta ya cuenta.
2. Si bien, el desarrollo de programas de IoT para mejorar la trazabilidad de procesos no es nueva en la empresa, en general, ya que se ha usado en la planta canadiense, ha habido retos importantes para obtener permisos y apoyo para implementar estas soluciones. Más específicamente la conexión Wifi de los dispositivos, y su integración con el ERP de la empresa, requiere de autorizaciones de agentes ajenos al desarrollo.

Dentro de la industria el internet de las cosas industrial y la aplicación de soluciones que implican Aprendizaje de Máquina, Visión Artificial y Conectividad entre dispositivos es un tema muy manido en los últimos años como señala Aceitón, 2020, quien explica que dentro de las integraciones logradas en la industria se identifica, dentro de sus arquitecturas, dos conjuntos de elementos; Los dispositivos de campo y elementos de control, como lo es el Controlador Lógico Programable (**PLC**), los Sistemas de Control Distribuido (**DCS**) y sistemas de Supervisión, Control y Adquisición de Datos (**SCADA**); Por otro lado, están los sistemas de administración, tales como Sistema de Ejecución de Manufactura (**MES**), y programas de Planificación de Recursos Empresariales (**ERP**). La integración de los sistemas de administración con los dispositivos de campo se ha utilizado para extraer datos de campo, transportarlos como información contextualizada hacia los niveles superiores para integrarla con la información administrativa y ayudar a la toma de decisiones de producción.

El PLC es un dispositivo que está próximo a recibir importantes transformaciones para adaptarse a las nuevas integraciones horizontales que pretenden incluir las necesidades de IoT junto con el control clásico que se ha manejado con los años. Empresas como OnLogic, Beckhoff, Siemens, etc. ya han lanzado al mercado múltiples opciones para esto. Sin embargo, Aceitón, 2020 señala un área de oportunidad que, de hecho, también se ha identificado en el transcurso del desarrollo del proyecto. Se trata de la imperiosa necesidad de incluir funciones de tecnologías operacionales (**OT**) junto con tecnologías de la información (**IT**). Las primeras incluyen funciones típicas de la industria para controlar equipos físicos como robots y actuadores. Su principal característica es que operan de manera determinística, con poco margen de latencia y con el uso de pocos recursos de memoria (unos cuantos kilobytes). En otro tenor, están las funciones de IT que son las que se encuentran en una computadora personal, las cuales no son determinísticas, y están enfocadas en la digitalización.

La integración de las funciones de IT en Mitchell Plastics Querétaro han ido de la mano de las Raspberry Pi 5 y los EWON de la marca HMS Networks que se han colocado alrededor de la planta. Mientras que las de OT se han implementado con PLC's de la marca Omron. Los retos de la empresa en este sentido han sido integrar los elementos administrativos de la empresa, como su ERP, PLEX, y los elementos de control, como lo son los PLC. El proyecto más avanzado en Canadá en esta área solamente utiliza la Raspberry Pi 5 para desplegar un dashboard programado previamente en un EWON, lo cual además de ser redundante, puesto que ambos son dispositivos de IoT que se pueden reemplazar mutuamente ¹, deja de fuera funcionalidades de la Raspberry Pi 5 y se desaprovechan las posibilidades de software que en este proyecto se explorarán.

2.2. Elementos del Proceso

Antes de describir las herramientas teóricas que se emplearán para crear el sistema de monitoreo en la Raspberry Pi, es de gran ayuda revisar los elementos que pasan por el proceso de ensamble de la estación cuyo nombre es "400D Handle Final Assembly". El propósito de la estación es ensamblar cuatro elementos que darán como resultado final una manija como la que se ve en la Figura 2.1. Sin embargo, debe tomarse en cuenta que en el proyecto se busca principalmente el registrar y graficar los datos recabados de la estación a partir de la información dada por diferentes sensores que son los que se enlistan después. Por razones de secreto industrial, no se mostrarán diseños ni códigos específicos utilizados en el desarrollo del proyecto.

¹Los EWON tienen ciertas ventajas sobre la Raspberry Pi 5 en cuanto a que son más cercanos a un PLC con funciones de IT que a una PC como sería la Raspberry Pi 5. Sin embargo, su programación es muy rústica; utiliza el lenguaje de programación BASIC en algunos modelos, y requiere de licencias especiales para ciertas funciones. Aún con eso, esa opción no integra bases de datos locales



Figura 2.1: Manija ensamblada en la estación 400D, Autoria propia, desarrollado en Mitchell Plastics Queretaro

2.2.1. Cámara IV4 Keyence

Las IV4, tal como indica la página oficial de KeyenceKEYENCE, 2026a, utilizan herramientas de inteligencia artificial de alto rendimiento para detectar piezas y variaciones en ellas. Cuenta con diversas opciones de programación como clasificación por IA, detección de color o de detección de IA. En el proceso se usa para evaluar si las piezas están bien colocadas en un nido. Evalúa la posición de las piezas y determina si están en “OK” o “NG” (Bien colocadas, mal colocadas). En la Figura 2.2 se muestra un ejemplo de cómo se ve esta cámara.



Figura 2.2: Cámaras IV4 extraída de la página web de Keyence F. A. Corporation, 2016

2.2.2. Robot FANUC

El encargado principal de ensamblar el conjunto de piezas es un brazo robótico fanuc con ayuda del controlador R-30iB Mate Plus, según el manual de uso “HandlingTool Operations and Programming” F. A. Corporation, 2016 el robot **FANUC LR Mate 200 iD** es un robot de seis juntas. Para programarlo y moverlo manualmente es necesario usar un

“Teach iPendant”. En la Figura 2.3 se presenta este modelo de robot. Se diseñó una herramienta específica para el ensamble la cual es diferente a la que se muestra en el final de brazo de la imagen de ejemplo, pero no es posible mostrar el diseño por motivos de secreto industrial.



Figura 2.3: Ejemplo de robot LR Mate 200 iD extraída de <https://assets.robots.com>

2.2.3. Cámara Serie VS y Prueba Eléctrica

Keyence KEYENCE, 2026a menciona que las cámara serie VS cuentan con inteligencia artificial para inspecciones sencillas y profundas con imágenes de hasta 25 mega píxeles y con alta resolución. Permite, además, la conexión con dispositivos externos como lámparas de iluminación. En el proceso su tarea es inspeccionar tanto el color del marco de la manejilla como un marcado láser como el que se aprecia en la Figura . De forma similar a la IV4, esta cámara devolverá un valor de “OK” o “NG” para el marcado láser y el color.

Este dispositivo al igual que la cámara IV4 se programa utilizando fotos de entrenamiento OK y NG usando una “Master” como referencia. Principalmente se usó la herramienta de detección de IA que mide las variaciones que encuentre en cada captura, y clasifica las piezas con base a un umbral definido por los programadores. En la Figura 2.7 se aprecian a) la cámara que realiza la inspección del marcado láser de la parte interna de la manija, b) el marcado láser, el cual se aprecia como una flecha amarilla, c) el pequeño circuito que se coloca justo debajo del botón de la manija que se aprecia en la imagen en la parte superior derecha de la pieza.

Antes de la estación de ensambaldo final, misma a la que se adecuó este proyecto, hay una estación de atornilla-

do que tiene como finalidad ensamblar el botón de la imagen c) de la Figura 2.7 con el resto de la manija como se aprecia en la imagen d). La manija al apretarse por un pin, debe crear un puente eléctrico en los dos puntos de soldadura de la placa.



(a) Cámara VS



(b) Marcado Láser



(c) Botón con placa para la prueba eléctrica



(d) Ensamble de la parte superior

Figura 2.4: Elementos de inspección en la estación de ensamblado 400D. Autoria propia, desarrollado en Mitchell Plastics Queretaro

Si bien hay más elementos en la estación como banda, sensores fotoeléctricos, cilindros, escáner, dispensador de grasa, etc. Los que se acaban de describir se consideran los más relevantes para entender el proyecto, no obstante, se mencionarán brevemente en las secciones posteriores si es necesario. A continuación se presenta una descripción del proceso de ensamble que se ejecuta en la estación para cada pieza.

2.3. Secuencia de la Estación

1. El operador coloca cuatro piezas en un nido, estas son un resorte, un bisel, un clavo y el marco de la manija. Hay en total de seis modelos los cuales combinan diferentes modelos de piezas. En la Tabla 2.1 se pueden observar los diferentes modelos que el operador debe colocar. Cada pieza cuenta con un numero de serie específico por modelo, un color de bisel y un color de marco de la manija:

Tabla 2.1: Modelos que se ensamblan en la estación 400D

Número de Serie	Color de Bisel	Color de Marco
42351A01	Warm Silver	Misty Gray
42351A02	Warm Silver	Black
42351A03	Warm Silver	MD Brown
42351A04	Silver	MD Gray
42351A05	Silver	Black
42351A06	Silver	Green

2. Se realiza una primera inspección en el nido ubicado en una mesa indexadora como se muestra en la Figura 2.5.
Si la inspección evalúa que los componentes están bien colocados la mesa gira poniendo el nido del lado del robot. Si no, salta una alarma en la HMI de que no están bien los componentes.

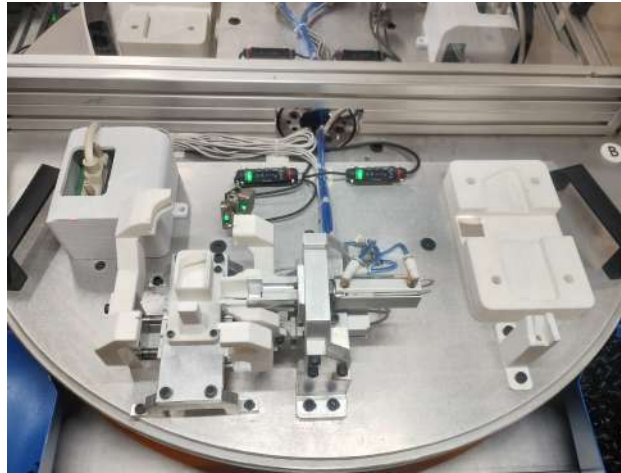


Figura 2.5: Nido donde se colocan las manijas sobre la mesa indexadora. Autoria propia, desarrollado en Mitchell Plastics Queretaro

3. El robot realiza el ensamble con la ayuda de unos cilindros en el nido. El robot estaba previamente programado para realizar esta acción.
4. Se realiza la inspección del marcado láser y el color de la manija usando la **Cámara VS**; el robot lleva la manija a una posición donde la cámara tenga buen ángulo y se toman automáticamente las capturas.
5. Pase o no la inspección de la VS se realiza una **Prueba Eléctrica**. La manija, además de jalarsse, puede presionarse como un botón. Dentro tiene una pequeña placa electrónica y se desea verificar que al presional el bisel se cree un puente entre dos nodos de la placa. En la estación hay un pin negro que sirve para que el robot presione el bisel ahí.
6. Si no pasa la prueba eléctrica o la inspección de la VS entonces el robot lleva la pieza a una banda transportadora

y esta la lleva a una caja de scrap. Si ambas inspecciones fueron OK entonces la banda transporta la manecilla hacia el operador.

7. Una impresora da una etiqueta IPL, que contiene información de la pieza y el operador debe colocarla en un lateral de la manija para al final registrarla con un escáner en la base de datos de la empresa.

2.4. PLC Omron NX1P2

Dentro de la empresa Mitchell Plastics, la principal herramienta que se usa para automatizar líneas de producción es el PLC de la marca japonesa Omron. Debe aclararse que, si bien existen PLC's con funciones de IoT integradas incluso de la misma marca, éstos suelen ser más caros y, además, sustituir todos los PLC que ya están operando en las líneas de producción es una tarea ardua y que provocaría un importante despilfarro de recursos. Por lo tanto, no se optó por eliminar lo que ya funciona sino complementarlo.

El PLC (Controlador Lógico Programable), es un ordenador especialmente diseñado para la operación en fábricas que puede comunicarse y operar con maquinaria industrial así como sensores y actuadores que, generalmente, operan con voltaje de 24 V.

Un PLC como cualquier otro ordenador se puede programar, pero, en el caso de estos dispositivos es posible hacerlo mediante diferentes lenguajes en un mismo programa. Según la IEC 61131 como se menciona en TecnoDigital, 2025, son cuatro los más utilizados:

- Diagrama de Escalera (LD): El más sencillo basado en esquemas eléctricos.
- Diagramas de Bloques de Función: Lógica a través de bloques conectados gráficamente
- Texto Estructurado (ST): Con una sintaxis tradicional parecida a otros lenguajes de programación secuencial.
- Lista de Instrucciones (IL) y Diagrama de Secuencias (SFC)

En el caso de este proyecto, el modelo de PLC que se utiliza en la estación de ensamblado de 400D es el PLC Omron NX1P2 (Véase la Figura 2.6) Según los manuales de Omron, el NX1P2 es un controlador con funciones avanzadas de control de movimiento y conectividad por medio del protocolo MQTT(Revisar la sección 2.5.2 de este documento), lo cual lo hace ideal para establecer comunicación con dispositivos de IoT. El NX1P2 cuenta con muchas otras características, pero lo importante para la ejecución del proyecto son las que se mencionaron ya que la programación y el sistema que se implementó en la estación es ajena a los objetivos de este proyecto.

Para realizar la comunicación es necesario utilizar el software **Sysmac Studio**. En este programa se usan los cuatro lenguajes anteriormente mencionados para implementar la secuencia de la estación de ensamblado, comunicarse con cámaras inteligentes IV4 y VS (por medio de mapeo de bits), y comunicarse mediante ethernet/IP y MQTT con la Raspberry que se agregará a la estación.



Figura 2.6: PLC Omron NX1P2. Extraído de: <https://www.ia.omron.com/products/family/3650/>

2.5. Industria 4.0

El presente proyecto tiene como finalidad adentrar a Mitchell Plastics Querétaro en tecnologías más modernas que las que se han estado usando hasta ahora. Según la revista “American Industrial Magazine” KEYENCE, 2026b la industria 4.0 integra tecnologías digitales inteligentes dentro de procesos productivos para crear fábricas conectadas, autónomas y basadas en datos. Dentro de las herramientas que se usan en ella son:

- IoT con sensores inteligentes.
- Inteligencia Artificial.
- Big Data.
- Integración MES–ERP.
- Simulación digital (gemelos digitales).

En este proyecto se consideró la ejecución de cuatro niveles como se muestran en la Tabla 2.2:

Tabla 2.2: Niveles de la industria 4.0

Nivel	Descripción	Cómo se implementa
1. Visibilidad	Ver qué está pasando	Panel de control con Grafana en tiempo real
2. Transparencia	Entender por qué pasa	Email reports y alertas automáticas.
3. Predicción	Anticipar qué va a pasar	Alertas predictivas basadas en tendencias
4. Autonomía	El sistema se corrige sólo	El sistema toma acciones automáticas

Debe considerarse que el proyecto inicial en sí sólo consta de implementar el nivel 1. Pero ello no limita la consecución de mejoras y avances en el futuro en las siguientes etapas de desarrollo. En este caso se espera integrar varias funciones de IT dentro de la estación de ensamblado como lo es la administración de bases de datos, la visualización de datos por medio de paneles de control y el uso de programación no determinística.

2.5.1. IoT con Raspberry Pi 5

La Raspberry Pi 5 es una mini computadora con un procesador Arm Cortex-A76 de cuatro núcleos y 64 bits que cuenta con memoria RAM desde 1 hasta 16GB. Además cuenta con GPU a través del VideoCore VII que le permite mostrar un entorno gráfico de 4k usando un HDMI según el documento oficial de [pip.raspberrypi.com](https://www.raspberrypi.com/documentation/hardware/raspberry_pi_5/) raspberrypi.com, 2026. Se ha elegido este dispositivo ya que cuenta con las suficientes capacidades para gestionar una base de datos, programar una lógica de eventos, usar ethernet ip para conectarse con el PLC, y para monitorear datos en tiempo real.

Parafraseando a Gokhale y Bhat Pradyumna et al., 2018 IoT se entiende como la red de objetos físicos como (dispositivos, instrumentos, vehículos, construcciones) y elementos embebidos con electrónica, software y sensores con conectividad que le permite a estos objetos reclectar e intercambiar información. El IoT permite a los dispositivos actuar sin intervención humana. Particularmente, en este proyecto se hará uso del IoT para crear la visibilidad de eventos en la estación de ensamble de las manijas.

Se comunican **Camara VS, PLC Omron NX1P2, y Raspberry pi 5** a través de diferentes protocolos de comunicación. Dado que la Raspberry Pi 5 funge como una microcomputadora, en ella es donde se pueden ejecutar programas para gestionar bases de datos, enviar correos electrónicos automáticamente y transmitir un panel de control por medio de HDMI, tareas senciles para el proyecto. Como se ve en la Figura 2.7, la Raspberry Pi 5 cuenta con un puerto Ethernet el cual servirá para realizar la conexión entre Raspberry Pi y PLC Omron. Asimismo cuenta con dos puertos micro HDMI que se conectarán a una pantalla para desplegar los paneles de control que se generarán a partir de la base de datos.

Las anteriores son las funciones que principalmente se usarán durante la ejecución del proyecto. No obstante hay muchas más: Soporte de IA, Visión Artificial, GPIO para señales de entrada y de salida, etc. Estas funciones, sin

embargo, serán exploradas por la empresa en próximos proyectos.

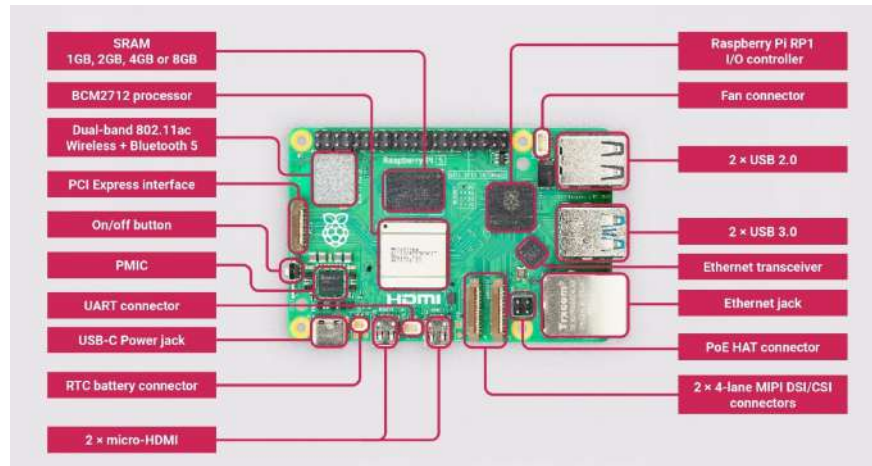


Figura 2.7: Raspberry Pi 5, y sus componentes. Extraído de: <https://www.hackatronic.com/raspberry-pi-5-pinout-specifications-pricing-a-complete-guide/>

2.6. Protocolos de Comunicación

2.6.1. Ethernet/IP

Según la descripción dada por Rockwell Automation Automation, 2000 el ^{et}hernet industrial protocol.^{es} un estándar para la interconexión de redes industriales. Utiliza medios físicos y chips comerciales capaz de gestionar hasta 1500 bytes por paquete en la transmisión de mensajes además de manejar los datos a una velocidad de hasta 100 Mbps.

En Mitchell Plastics se manejan direcciones IPv4 para los diferentes dispositivos dentro de la planta. Como señala Jonas P. Lüke Lüke, 2019 las direcciones IP de versión 4 son un entero de 32 bits separados en 4 bytes. Para un correcto funcionamiento de la red, todo dispositivo debe tener asignado una dirección IP diferente a la de los demás dispositivos dentro de esa misma red. La dirección IP se compone de dos partes:

- Prefijo de red: Identifica a la red en general.
- Host: Identifica a un elemento concreto dentro de la red.

En este caso, la Raspberry Pi 5 tiene una dirección IP específica que sólo está presente cuando hay conexión por medio de cable RJ45 a un hub de conexiones ethernet.

En la estación de ensamble se maneja el siguiente esquema (Figura 2.8):

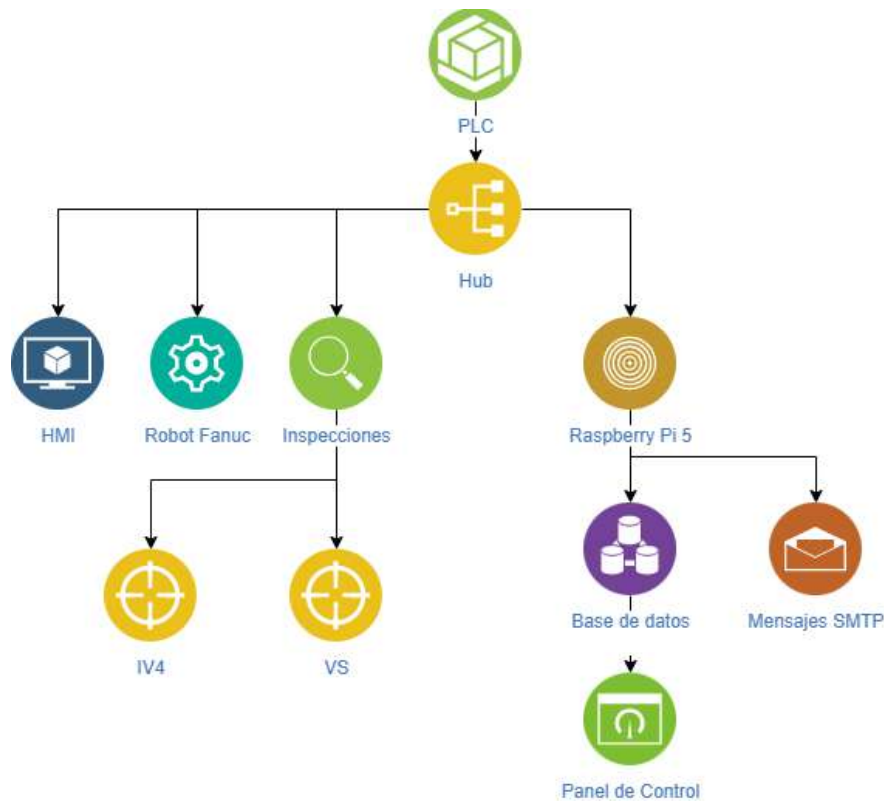


Figura 2.8: Conexiones por Ethernet/IP en el proyecto. Autoria propia, desarrollado en Michell Plastics Queretaro

Todos los dispositivos que se muestran en el esquema, incluyendo al PLC, están conectados mediante cables con conectores RJ45 Tipo B (véase la Figura 2.9) a un Hub o Switch de puertos ethernet. Estos dispositivos incluyen la HMI (una pantalla touch marca Keyence), el robot marca FANUC, las cámaras inteligentes IV4 y VS, y finalmente la Raspberry Pi 5. Todos estos dispositivos se comunican mediante Ethernet/IP, cada uno se configura de manera diferente para establecer adecuadamente la conexión con el PLC².

En realidad, no todos los dispositivos operan con ethernet/ip dentro de la estación de ensamblado del proyecto. Pero si es la principal herramienta que se usó para el proyecto en sí. Por otro lado, también se utilizaron mucho los **puertos de red**. Son esenciales para el proyecto ya que es un identificador único que permite que múltiples programas operen con la misma dirección IP. Operan como un punto de acceso al que pueden enviar o leer información diferentes programas. Por ejemplo, para una base de datos, en un dispositivo con dirección 192.150.250.40, ella puede realizar sus servicios en el puerto 5050. De ese modo, si por medio de una computadora o dispositivo externo que entre a la red, se accede a la dirección 192.150.250.40:5050 en el navegador podrá acceder a la base de datos dentro del dispositivo host (El host funge como servidor).

²Todos, salvo la raspberry, requieren un mapeo de bits que se puede obtener de los manuales oficiales de cada dispositivo de la marca Keyence.

Conforme se avance en el desarrollo de este documento, se verá cómo se han configurado las direcciones IP y los puertos necesarios para los diferentes programas.

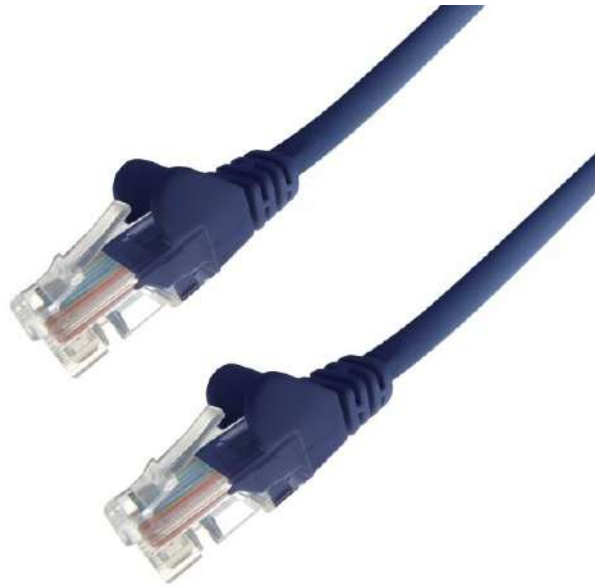


Figura 2.9: Cables con conectores RJ45 para conexiones con PLC. Imagen obtenida de: <https://www.smart-secure.co.uk/product/0-3m-rj45-cat6-utp-network-cable-blue/>

Tanto los dispositivos físicos como el servidor local de la base de datos en PostgreSQL como el panel de control en Grafana requieren definir una dirección IP en la que operan dentro de la red y un puerto de acceso. En la sección del desarrollo se especificará cómo y cuál es esa definición para cada uno. En el caso de la Raspberry se requiere usar otro protocolo para la transmisión de la información desde el PLC hasta la misma. A continuación, se presenta.

2.6.2. MQTT

Como indican Dipa Soni y Ashwin Makwana Soni y Makwana, 2017, lanzado por IBM en 1999, el protocolo Message Queue Telemetry Transport (transporte de telemetría por colas de mensajes) o MQTT por sus siglas en inglés, es un protocolo de estandarizado de publicación/subscripción que transmite mensajes por medio de paquetes de control. Sus características básicas son las siguientes:

- **Publicación/Subscripción:** En MQTT, hay clientes con dos funciones esenciales; publicar y subscribirse. Los clientes pueden publicar mensajes (cadenas de texto) en un *Topic*, una especie de dirección específica escrita con el patrón /(nombre del tópico). Aquellos clientes que se subscriban al tópico podrán leer todos los mensajes que lleguen a él desde cualquier otro cliente.
- **Tópicos y subscripciones:** Se pueden definir múltiples tópicos y subtópicos (tópicos dentro de tópicos) en

MQTT para diferenciar los mensajes que se estarán recibiendo en cada uno. Además, en la subscripción es posible especificar parámetros de seguridad. En este proyecto no se usaron estos últimos.

- **Quality of Service Levels (QoS):** Los clientes pueden establecer cuál será el nivel de calidad de la distribución de los datos recibidos y enviados. Concretamente, en MQTT hay tres niveles:

1. El mensaje solo se envía una única vez.
2. El mensaje se envía mínimo en una ocasión, pero se puede configurar para enviar el mismo mensaje varias veces.
3. El mensaje se envía en una ocasión, pero con una encriptación de seguridad llamada “4-way handshake”.

En el proyecto solo se usará el nivel 1.

Para controlar la distribución de información de MQTT se utiliza un *Broker*. Este Broker es el principal responsable de aceptar requerimientos de clientes, recibir mensajes publicados por usuarios, enviar los mensajes desde el publicador hasta los usuarios interesados, etc. En este caso se hará uso del broker “Mosquitto” el cual es de código abierto. Como indica la página web de logicbus Logicbus, 2024, este tiene alta compatibilidad con computadoras de baja gama así como con servidores industriales, además de tener un consumo bajo de datos.

2.6.3. SMTP

SMTP son las siglas para Simple Mail Transfer Protocol. Como lo indica Vladimir V. Riabov, en la parte 3, p.878 Bidgoli, 2006 es un mecanismo de transporte electrónico de mails en el que un cliente SMTP abre una conexión TCP a un servidor SMTP (en este proyecto se trata de un servidor empresarial) en algún puerto específico (usualmente 25). Si la conexión es exitosa ambos procedimientos SMTP del servidor y del cliente inician un diálogo donde el cliente proporciona las direcciones de origen y de destinatario del correo para posteriormente enviarlo.

Los correos electrónicos son enviados mediante por una serie de transacciones de tipo requerimiento-respuesta(request-response) entre los clientes y el servidor. Una transacción de este tipo debe contar con “headers” los cuales especifican formato, destinatario, origen, servidor, etc. Es información útil para indicar cómo se debe enviar el mensaje. El cuerpo del mensaje será lo que se mostrará en el email para el destinatario.

2.6.4. JSON y Diccionarios

Los diferentes protocolos de comunicación que se utilizan en este proyecto y algunos programas que se verán más adelante, requieren de una estructura de datos específica que será útil para enviar información entre dispositivos y entre

programas. Esto es, hasta ahora se han revisado los canales, puentes o métodos de comunicación, pero se debe revisar el vehículo principal de esa misma comunicación. El JSON y el Diccionario (también llamado hash table, object o map). Aunque son conceptos muy relacionados, son bastante diferentes.

Por un lado, el **JSON (JavaScript Object Notation)** es un estándar que como indica Tom Marrs Marrs, 2017, no es una estructura de datos sino una cadena de texto con un formato o sintaxis especial. Concretamente se construye mediante el estándar:

{CLAVE:VALOR}

Como se trata de una cadena de texto, es fácil para muchos lenguajes de programación procesarla y operar con ella para extraer los datos que en el JSON se contengan. El JSON debe cumplir con las siguientes características según Tom Marrs Marrs, 2017:

- Cada clave debe estar al lado izquierdo de dos puntos (:) y debe ser una cadena de texto entre comillas.
- El valor que se asigna a la clave puede ser cualquier tipo de dato (recordar que el JSON es un texto, pero el formato final de los valores del JSON puede cambiar a entero, arreglo, texto, etc. cuando se pase a una hash table como se verá más adelante) y debe ir al lado derecho de los dos puntos.

En la Figura 2.1 se puede observar un ejemplo de JSON.

```
{  
  "array": [  
    1,  
    2,  
    3  
  ],  
  "boolean": true,  
  "color": "gold",  
  "null": null,  
  "number": 123,  
  "object": {  
    "a": "b",  
    "c": "d"  
  }  
}
```

Figura 2.1: JSON cuyos valores varían en formato.

Por otra parte, el diccionario o hash table es una estructura de datos que, si bien tiene un formato idéntico al JSON, ésta cuenta con una estructura específica en la memoria de la computadora, lo que le permite a la misma acceder a los valores de forma muy rápida y eficiente. Cada lenguaje de programación opera de forma distinta sobre esta estructura de datos, pero, en general, se utilizan funciones para acceder a los valores por medio de indexación. Como se muestra en la Figura 2.10. En este proyecto se usa JSON para enviar los datos a la Raspberry pi y hash table para procesar los datos y enviarlos a la base de datos y de ahí a los reportes de Email.

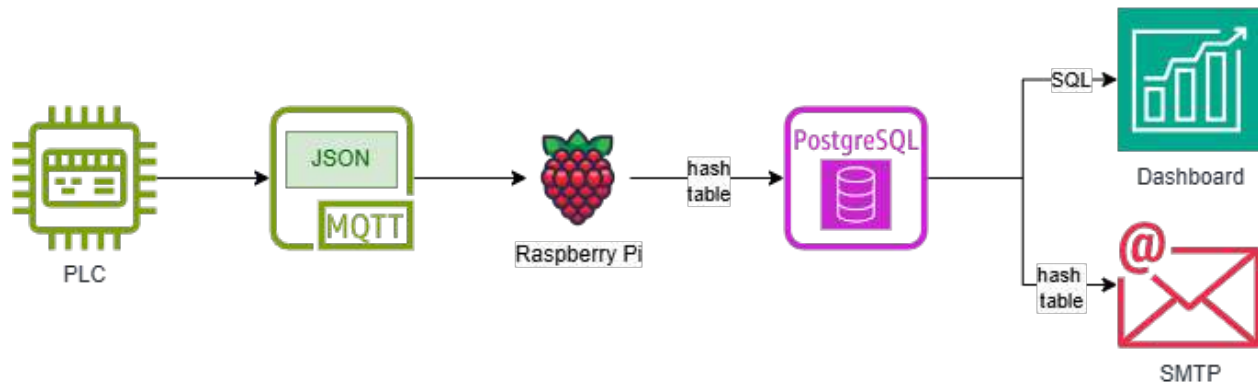


Figura 2.10: Comunicaciones en el proyecto utilizando JSON y hash tables. Autoría propia, desarrollado en Mitchell Plastics Queretaro

Conforme se avance en la sección de desarrollo se ahondará en cómo se usan estos elementos dentro de la programación del proyecto.

2.7. Software Utilizado

2.7.1. Debian Raspberry Pi Linux

El sistema operativo con el que se trabajó dentro de la Raspberry Pi 5 fue Linux, concretamente la distribución Debian. Rumbos Salomón Salomón, 2012 menciona que se trata de un proyecto de software libre. Para este proyecto fue necesario utilizar la línea de comandos de este sistema operativo para lograr que todos los programas, salvo Sysmac Studio, que se revisarán posteriormente en esta sección funcionen de acuerdo a las necesidades del proyecto.

La principal herramienta que se debe tomar en cuenta en este proyecto es la terminal de Debian. En ella es donde se configuran todos los programas a través de comandos. Conforme se avance en el desarrollo de este documento se mencionarán algunos de ellos según sea la necesidad.

Rumbos Salomón, 2012 indica que todo lo que puedes hacer en un entorno gráfico lo puedes hacer utilizando la

terminal de debian linux; Crear, eliminar o mover archivos; Monitorear estadísticas del sistema como temperatura, programas en ejecución, estatus de conexiones; Ejecutar programas, o programarlos para que se ejecuten al encender; editar el entorno gráfico cambiando apariencia, cursor, tamaño y fuentes de texto; y muchas más características. Es decir, su funcionalidad va más allá de instalar y actualizar programas en este proyecto. Esto último debido a que hay ciertos requerimientos que se deben adaptar para la operación en planta. Por ejemplo, el panel de control debe estar siempre activo independientemente de que la estación se apague y, por lo tanto deje de existir la dirección IP de la conexión ethernet, o bien, debe configurarse cómo debe responder la Raspberry en el caso de que se pierda la conexión WiFi.

Aquí debe tenerse en cuenta que, en realidad, no se utiliza la versión Debian típica de uso personal que se suele instalar en PC's, sino una versión adaptada al microcontrolador. Sin embargo, comparte con Debian comandos similares y al igual que muchas otras distribuciones, contiene la shell *BASH* por sus siglas en inglés Bourne-Again Shell³. Como señala Shotts, 2019, una shell, o terminal como se referirá en este documento, es un programa cuya tarea es tomar comandos pasados por teclado y enviárselos al sistema operativo para que este ejecute ciertas acciones. Los comandos serán el medio principal por el que se harán muchas de las configuraciones que son esenciales para que los programas que aquí se usan puedan operar.

Linux opera por medio de directorios, es decir, archivos que contienen los programas que se usan en el sistema. Tiene un árbol de jerarquía que organiza el sistema, y a diferencia de el explorador de archivos de windows, en Linux no hay extensiones de archivos, es decir para el sistema, le es indiferente si archivo finaliza con .py, .txt o .bmp. Los directorios fundamentales que se usan en este proyecto son los siguientes:

Tabla 2.3: Directorios de utilidad de Linux.

Directorio	Descripción
/	Es el directorio de mayor jerarquía del sistema
/bin	Contiene programas presentes para que el sistema pueda correr.
/etc	Uno de los más importantes pues ahí se almacenan archivos de configuración y scripts automáticos
/home	Un directorio exclusivo para un usuario y por lo tanto seguro para el sistema.

Cabe recalcar que el sistema operativo fue modificado para establecer conexión con el Wifi de la empresa, pero ese tema fue hecho con apoyo del departamento de IT por lo que no forma parte de lo desarrollado en este documento.

³Linux desde su creación tiene una historia de desarrollo enorme que se recomienda encarecidamente explorar. En este caso, el nombre BASH hace referencia a Steve Bourne, quien desarrolló la shell predecesora perteneciente a UNIX antes de la creación de Linux llamada sh.

2.7.2. Sysmac Studio

Sysmac Studio es el entorno de desarrollo integrado que la empresa Omron provee para desarrollar programas para los PLC modelos NX entre otros, según el portal de ventas de Omron O. Corporation, 2026 este programa cumple en su totalidad el estándar abierto IEC 61131 el cual se había mencionado anteriormente. Para utilizarlo legalmente es necesario haber adquirido una licencia. Es menester mencionar que la empresa cuenta con dicha licencia para acceso al software para el área de automatización. En la Figura 2.11 se muestra cómo es la interfaz gráfica del software (Solo es un ejemplo disponible en la página oficial de omron, no tiene código propio de la empresa).



Figura 2.11: Interfaz gráfica de Sysmac Studio Extraída de <https://industrial.omron.es/es/products/sysmac-studio#features>

Este es el primer programa con el que se harán algunas modificaciones a la estación. Dicho sea de paso, se pueden instalar librerías dentro del software para tareas específicas; en este proyecto se instaló una librería para manejar MQTT. Aquí cabe señalar que el modelo de PLC utilizado no puede instalar un MQTT broker (Revisar sección 2.5.2), pero sí puede fungir como cliente para conectarse a un tópico de MQTT usando la librería MQTT versión 0.45 disponible para Sysmac Studio

2.7.3. Node-RED

Como lo mencionan Firas Ahmed Hussein, entre otros Hussein et al., 2025 Node-RED es una herramienta de programación visual basada en flujos que ayuda a conectar y automatizar servicios de internet, dispositivos de IoT, y API's. Es un programa construido sobre el lenguaje de programación JavaScript y en la plataforma Node.js. En múltiples estudios según se señala en Hussein et al., 2025 se ha utilizado Node-RED para aplicaciones de IoT para control de sistemas, monitoreo en tiempo real, sistemas SCADA, y particularmente con MQTT para enviar y recibir datos de sensores y actuadores. Esta aplicación se presenta como una opción preferible a lenguajes de programación de bajo nivel, ya que

permite ejecutar acciones complejas en diagramas de flujo simples. Además, al ser una herramienta de código abierto, tiene una comunidad grande que ha desarrollado múltiples librerías para las necesidades de integración con software que está surgiendo constantemente. En la Figura 2.12 se puede apreciar un flujo de trabajo que no pertenece a Mitchell Plastics dentro de Node-RED.

Como se puede apreciar ahí, La lógica consiste en insertar nodos en un flujo y conectarlos para formar ramas. Se dice que Node-RED es programación basada en eventos ya que si algún nodo se activa bajo una condición puede hacer que fluya la información a través de toda una rama del flujo. Algunos nodos pueden ejecutar scripts enteros programados en JavaScript u otros lenguajes, y pueden crearse paneles de control con las variables dentro de Node-RED. Como aclaración, los paneles de monitoreo que se desarrollarán en este proyecto no están hechos en Node-RED, sino en otro software llamado grafana que se verá posteriormente.

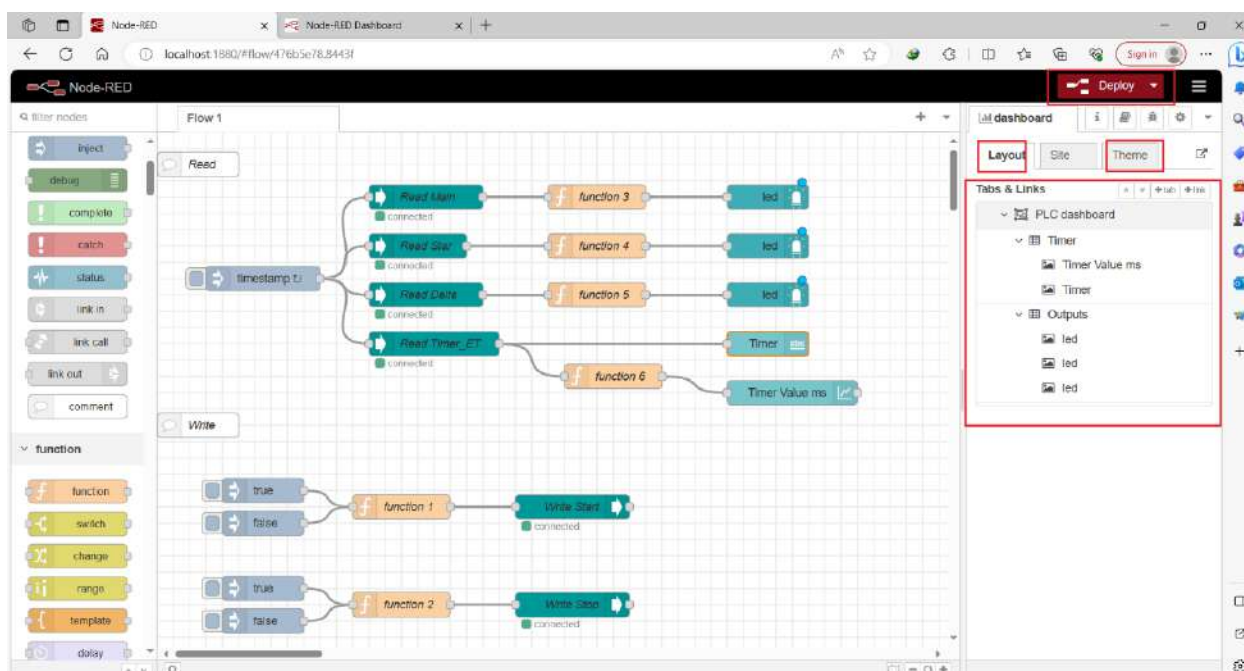


Figura 2.12: Flujo de ejemplo de Node-RED, extraído de <https://www.solisplc.com/tutorials/siemens-tia-portal-plc-dashboard-using-node-red-a-step-by-step-tutorial>

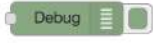
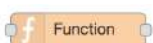
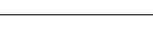
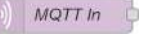
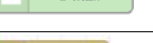
En la siguiente Tabla 2.4 pueden apreciarse algunos de los nodos que se estarán utilizando en los programas del proyecto. Muchos de ellos no son nativos de Node-RED, por lo tanto, se necesitan descargar librerías para utilizarlos. Los nodos pueden tener entrada de datos y salida de datos, conforme se activan los flujos, los nodos pueden intercambiar datos de unos a otros siguiendo las conexiones (líneas grises) que se crean en el programa.

Cabe señalar que en Node-RED se pueden configurar los nodos de diferentes maneras incluyendo su apariencia e

incluso cambiar la programación interna de los mismos o crear nodos completamente nuevos. Si se ha cambiado algún nodo para tener una apariencia o función distinta para algún programa específico, se mencionará en la sección de desarrollo de este documento.

También es posible instalar librerías extra dentro de los propios nodos o editar sus propiedades a partir del código JSON que genera Node-RED para los nodos. Node-RED debe correr como un servicio dentro del sistema de la Raspberry Pi 5 en este proyecto. Comúnmente se abre dentro del navegador poniendo la dirección IP en la barra de búsqueda junto con el puerto de acceso que por defecto, al instalar Node-RED, es el puerto 1880. Si bien, el programa se ejecuta en la terminal de linux de la Raspberry, al conectarse a la misma red Ethernet/IP, cualquier dispositivo con permiso puede acceder remotamente al programa de Node-RED desde su propio navegador sin tener que descargar Node-RED de nuevo.

Tabla 2.4: Nodos que se usarán para desarrollar el proyecto. Gran parte de la información se obtuvo de la página oficial de Node-RED O’Leary y Conway-Jones, 2026.

Imagen	Nombre	Descripción
	Inject	Se usa para activar manualmente un flujo. Puede configurarse para enviar la señal de activación por intervalos, a cierta hora, etc.
	Debug	Permite enviar mensajes a la pestaña de depuración de Node-RED.
	Function	En este nodo se añade código de JavaScript. El código puede operar con los datos que recibe el nodo y posteriormente enviar nuevos datos a los siguientes nodos.
	Switch	Evalúa una condición para decidir si el flujo continúa o se detiene en este nodo. Se le puede configurar con múltiples salidas para múltiples condiciones.
	node-red-contrib-postgresql	En este nodo se realiza la conexión con una base de datos de PostgreSQL y también envía queries a la misma.
	MQTT in	Permite subscribirse a un tópico de MQTT y obtener todos los mensajes que ahí se publiquen.
	node-red-node-email	Establece conexión con un servidor y envía correos electrónicos con el protocolo SMTP.
	CSV	Permite formatear los datos recibidos en la entrada a un archivo CSV.

2.7.4. JavaScript

Para programar los diferentes nodos de Node-RED es posible utilizar herramientas de configuración predeterminadas, pero en muchas ocasiones también es necesario crear un script para tener más control sobre lo que hace el nodo y el flujo al que pertenece en su totalidad. El lenguaje de programación predeterminado para estas tareas es JavaScript. En

su libro Svekis et al., 2021 definen a JavaScript como un lenguaje interpretado lanzado en 1995 que se ha convertido en uno de los más usados para navegadores web y otras aplicaciones, principalmente porque estos navegadores pueden interpretarlo y, generalmente, viene instalado en el software de la computadora por defecto. Además su uso suele ir de la mano con herramientas de desarrollo web como HTML como se verá en este proyecto.

Si bien, no se adentrará en este documento en profundidad en todas las funciones que tiene JavaScript, sí que se dará un vistazo a algunas de las herramientas que se aprovecharon para el proyecto. La más utilizada es la estructura “Object” de JavaScript. Svekis et al., 2021 indican que el “Object” es una manera de almacenar múltiples variables dentro de una sola. Anteriormente se explicó qué es un diccionario en el contexto de la programación. Un “Object” es un diccionario o “hash table” que se puede operar con diferentes métodos y funciones para acceder rápidamente a las variables que contiene. Svekis et al., 2021 aportan el siguiente ejemplo:

```
let dog = { dogName: "JavaScript",  
weight: 2.4,  
color: "brown",  
breed: "chihuahua",  
age: 3,  
burglarBiter: true  
};
```

Figura 2.2: Ejemplo de Object en JavaScript extraído de Svekis et al., 2021

Las diferentes variables del objeto creado son accesibles por medio de dos métodos. Como se ve en la Figura 2.3

```
1 let dogColor1 = dog["color"];  
2 let dogColor2 = dog.color;
```

Figura 2.3: Accesos a variables dentro de un Object extraído de Svekis et al., 2021

Todo valor de la estructura puede ser actualizado para tener otro valor. Esto es de gran utilidad, puesto que en Node-RED la información que viaja entre nodos es un objeto llamado msg; ese objeto transporta los resultados de la ejecución de un nodo a otro nodo completamente diferente.⁴ Desde Luego, JavaScript cuenta con sus propias estructuras condicionales, bucles, funciones especiales, etc., pero se irán mencionando en este documento si es necesario.

⁴Para más información sobre JavaScript se recomienda consultar ya sea el libro de Svekis et al., 2021 u otros elementos en la web sobre el lenguaje de programación JavaScript como el curso de freecodecamp disponible en internet.

2.7.5. HTML

Casado-Vara, 2019 explica que HTML es un lenguaje que permite describir hipertexto, es decir, texto estructurado con vínculos, enlaces y elementos multimedia para generar páginas web interactivas. Un documento HTML esta compuesto por elementos que representan estructuras o comportamientos deseados, como por ejemplo, párrafos, vínculos, listas, etc. Una declaración de un elemento consta generalmente de tres partes: etiqueta inicial, contenido y etiqueta final.

La estructura de un documento HTML contiene tres partes

1. Una línea que contiene información de la versión HTML.
2. Un cuerpo del documento delimitado por el elemento <body>el cual será el contenido principal que se mostrará en la pagina web o programa en que se ejecute el código HTML.
3. Una sección dedicada a scripting.

Véase el siguiente ejemplo:

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="utf-8" />
5     <meta
6       name="viewport"
7       content="width=device-width, initial-scale=1.0" />
8     <title>freeCodeCamp</title>
9     <link rel="stylesheet" href="./styles.css" />
10  </head>
11  <body>
12    <h1>freeCodeCamp</h1>
13    <p>
14      freeCodeCamp.org is a non-profit organization that consists of an interactive
15      learning web platform, an online community forum, chat rooms, and local
16      organizations that intend to make learning web development accessible to
17      anyone.
18    </p>
19  </body>
20 </html>
```

Figura 2.4: Estructura de un documento HTML. Elaboracion propia label

En este proyecto, HTML se utilizará para hacer envíos de reportes por email con estilos atractivos y, además, será de utilidad para editar algunas gráficas dentro de grafana. Debe considerarse que en ambos casos se utilizó Inline CSS para brindar a los elementos de HTML mayor estética. Inline CSS es un método dentro de HTML que permite definir las características como color, tamaño, alineación, etc. En la Figura 2.5 se muestra cómo se usa.

```
1 <h1 style="color: blue; font-size: 24px; text-align: center;">
2   Esto es un titulo estilizado.
3 </h1>
```

Figura 2.5: Uso de Inline CSS. Elaboracion propia

Algunas reglas de HTML que se usan en el proyecto son las siguientes mencionadas por Casado-Vara, 2019:

1. Algunos tipos de elementos permiten a los autores omitir las etiquetas finales (por ejemplo <p>y).
2. Otros tipos de elementos permiten omitir las herramientas iniciales (<head>y <body>).
3. Existen otros elementos, como por ejemplo
que no tiene contenido y su único papel es determinar un salto de línea.

2.7.6. PostgreSQL

Una de las funciones protagonistas de este proyecto es la capacidad de integrar los datos de trazabilidad en una base de datos, es decir, tener una colección de información estructurada sobre las variables del proceso.

Para administrar una base de datos se suele usar un Sistema Gestor de Base de Datos (SGBD) que permita realizar las tareas de programación de forma sencilla e incluso visual.

En este caso se optó por hacer uso de PostgreSQL, como indica la documentación oficial de PostgreSQL de la versión 18.4 Group, 2026 define a este programa como un sistema de gestión de bases de datos objeto-relacionales de uso libre. Algunas características que comparte la documentación sobre el programa son las siguientes:

- Consultas Complejas.
- Llaves Foráneas.

- Triggers (desencadenantes de eventos).
- Vistas Adaptables.
- Integridad Transaccional.
- Control de Concurrencia Multiversión.

Además, el usuario que use PostgreSQL puede añadir:

- Tipos de Datos.
- Funciones.
- Operadores.
- Funciones de Agregación.
- Lenguajes Procedurales.

PostgreSQL funciona con un modelo de cliente/servidor; el servidor se encarga de gestionar los archivos de las bases de datos y, por defecto, el programa del servidor se llama Postgres. El servidor puede gestionar quiénes acceden a las bases de datos a través de un puerto en la dirección IP del dispositivo donde este alojado.

En este proyecto, además de PostgreSQL se utilizó **pgAdmin 4**, una herramienta para gestionar la base de datos de manera visual. Al igual que el propio PostgreSQL es de código libre ya que de hecho es un entorno virtual de python. A este programa se puede acceder a través de la dirección IP del dispositivo dentro de cualquier navegador al igual que Node-RED. Asimismo, cuenta con su propio puerto de acceso, normalmente el 5050. En la Figura 2.13 se observa la interfaz gráfica de pgAdmin4, se puede ver una base de datos y una consulta a la misma por medio de la herramienta de consultas "Queries" del programa.

PostgreSQL está diseñado para tener millones de registros en poco tiempo, y es preferible utilizar un programa SQL para la gestión de base de datos en lugar de otros como Excel por dos razones principales:

- Detrás de postgresql corren programas de álgebra relacional que aumentan el rendimiento a la hora de hacer consultas a la base de datos.
- Excel es un software de pago y además requiere muchos recursos computacionales que la raspberry pi no es capaz de suplir.
- Excel se vuelve lento conforme crece la base de datos, postgresql u otros programas similares no.

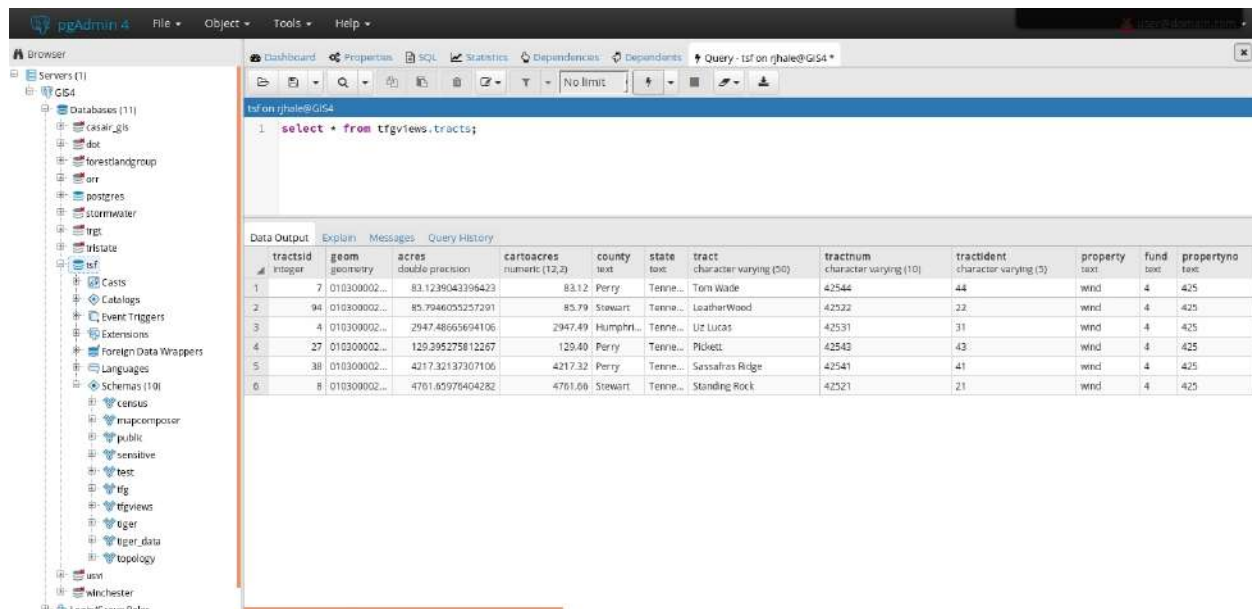


Figura 2.13: Interfaz Gráfica de pgAdmin 4 Extraída de <https://www.northrivergeographic.com/adventures-in-pgadmin4/>

PostgreSQL aprovecha el Lenguaje de Consulta Estructurada (SQL por sus siglas en inglés) que según la documentación Group, 2026 es el lenguaje estándar para comunicación, manipulación y gestión de bases de datos. Este lenguaje funciona mediante “SQL Queries” o comandos SQL que describen qué es lo que quiere el usuario que haga el programa, haciendo de este un lenguaje *declarativo*. Sin embargo, PostgreSQL tiene algunas funciones extra a las del SQL usado en otros programas. Algunas de ellas se verán cuando se pase al desarrollo del proyecto, pero cabe mencionar que PostgreSQL es un software bastante potente para las tareas que se requieren, esto tiene la contraparte de que es un poco pesado para la Raspberry Pi 5, para este proyecto no ha provocado errores, pero el desempeño del proyecto se explorará a detalle en la parte de resultados de este documento.

La documentación Group, 2026 proporciona la Figura 3.2 para crear una tabla llamada *weather* en una base de datos para comprender mejor estos conceptos. Se refiere a que SQL es declarativo en el sentido en el que el código solamente le dice al programa qué quiere que pase, en este caso que se cree una tabla, en lugar de cómo hacerlo.

```
CREATE TABLE weather (
    city varchar(80),
    temp_lo int, -- low temperature
    temp_hi int, -- high temperature
    prcp real, -- precipitation
    date date
```

);

Figura 2.6: Creación de una tabla con SQL. Elaboracion propia

Como se puede ver en el ejemplo, una tabla dentro de una base de datos está conformada por filas y columnas; Cada fila es un registro hecho en la base de datos y es común aunque no obligatorio usar una llave primaria o un identificador único para cada fila. Así, se tiene la certeza de que no hay registros duplicados. Por otra parte cada columna representa un atributo del registro. Las columnas pueden tener diferentes tipos de datos ⁵ (int, text, date, datetime, JSON, array); restricciones como que una columna de enteros nunca pase de cierto valor límite; y otras configuraciones como que se genere automaticamente.

Otra característica clave para el proyecto de PostgreSQL, y del lenguaje SQL en general, son las **vistas** (Views). La documentación Group, 2026 menciona que no es más que una consulta a la que se le da un nombre específico. En lugar de escribir una consulta cada vez que se quiera revisar algunos datos particulares, por ejemplo, la producción del día actual, el conteo de piezas por turno, etc. Se puede crear una vista con la consulta almacenada y acudir a ella en todo momento que sea necesario. La vista devolverá una tabla que puede ser manipulada como tal sin afectar la tabla principal de la base de datos. Crear múltiples vistas se considera un aspecto clave del diseño de una base de datos, y es normal hacer vistas sobre otras vistas. Las vistas, como se verá en el desarrollo de este proyecto, ayudan a extraer información particular de los datos generales de la base de datos.

2.7.7. Grafana

Una vez que los datos sean enviados desde el PLC a la Raspberry con MQTT para procesarlos y generar reportes por email en Node-RED, y almacenarlos con PostgreSQL, se procederá a visualizar la información. Para ello se usará Grafana.

Si bien, tanto Node-RED como pgAdmin 4 tienen funciones para generar gráficas o paneles de control. En este proyecto se optó por utilizar Grafana, una herramienta especializada para crear visualizaciones más sofisticadas.

Grafana es una herramienta de monitoreo, visualización y análisis de datos en tiempo real. Al igual que la mayoría de los programas mencionados en esta sección, grafana es software libre, lo que quiere decir que, además de no tener costo, tiene una comunidad grande que ha desarrollado muchas librerías para Grafana.

⁵PostgreSQL se caracteriza por manejar tipos de datos de gran complejidad como el JSON, XML o geométricos. En este proyecto no se usaron tipos de datos tan complejos, pero se recomienda investigar la documentación para más información de esos tipos de datos

Según el libro de Salituro, Eric Salituro, 2023 Grafana es un programa desarrollado con el lenguaje de programación *Go*. Grafana es capaz de recibir datos desde sistemas gestores de bases de datos tradicionales como MySQL o PostgreSQL, hasta más modernos y no relacionales como MongoDB, InfluxDB o Prometheus. Un mismo tablero en grafana no tiene por que limitarse a extraer datos de una sola fuente sino que puede combinar datos de diferentes fuentes.

Salituro Salituro, 2023 señala que Grafana está estructurado alrededor de tres componentes de interfaz de usuario principales; **paneles, tableros y filas**. El tablero será la pantalla principal que se cerán todos los paneles o gráficas que se hayan insertado en él. En la Figura 2.14 se puede apreciar un tablero de grafana con múltiples elementos, como se puede ver, el tablero (dashboard) es el elemento más general dentro de Grafana.

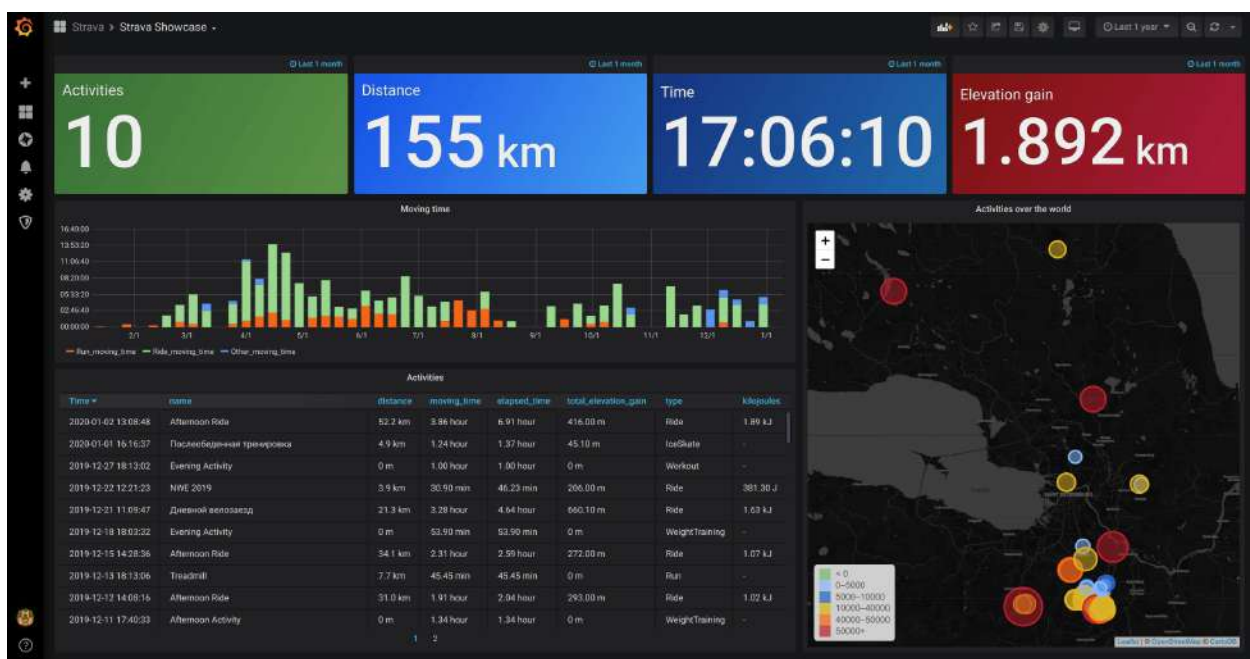


Figura 2.14: Panel de ejemplo en Grafana, extraído de: <https://logit.io/blog/post/top-grafana-dashboards-and-visualisations/>

Por otro lado, los paneles son las gráficas dentro del tablero, es decir, son las visualizaciones de los datos, y en ellos se realizan las consultas pertinentes para traducir los registros en la base de datos a gráficas interactivas. Salituro Salituro, 2023 agregó en su libro la Figura 2.15 de la interfaz gráfica de usuario para crear un panel.

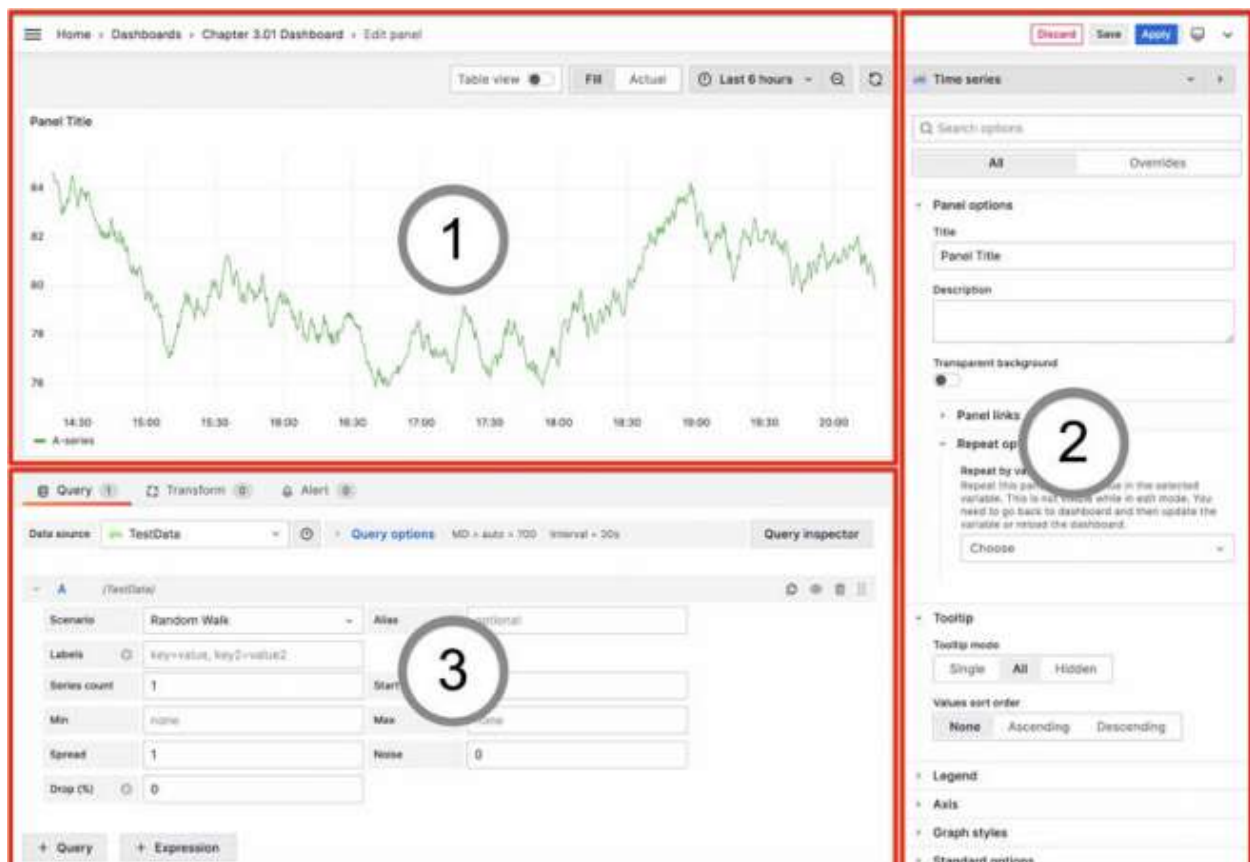


Figura 2.15: 1) Es la gráfica generada por el programa, 2) Opciones de Ajuste donde se define el tipo de visualización, estilos, fuentes, tamaños, etc., es decir, la configuración de la gráfica 3) Las tablas del panel donde se hacen las consultas a la base de datos. Extraído de <https://grafana.com/docs/grafana/latest/visualizations/panels-visualizations/visualizations/>

Por último las filas son los contenedores de los paneles. Sirven para organizar de manera gráfica los paneles haciendo click y arrastrando con el cursor facilitando así el desarrollo del tablero. Grafana puede configurarse para mostrar múltiples tableros en intervalos de tiempo cambiando la pantalla de visualización. A eso se le llama una “playlist” y se suele configurar en modo kiosco, es decir, mostrar los tableros en pantalla completo sin menús de edición.

Grafana es accesible por medio de la dirección IP del dispositivo donde esté ejecutándose. Se requiere un puerto de acceso, que, por defecto, es el puerto 3000 El tablero de los datos de la estación de ensamble será el resultado final del proyecto además de la base de datos funcional, y las alertas y reportes por correo electrónico. No todas las herramientas de grafana serán útiles para el proyecto y las que se usen se irán describiendo conforme se necesite en el capítulo 3.

Capítulo 3

Desarrollo

En el marco teórico ya se especificó cómo es la secuencia que se realiza en la estación de ensamblado donde se colocará la Raspberry del proyecto. En la Figura 3.1 se muestra de manera esquemática esa misma secuencia.

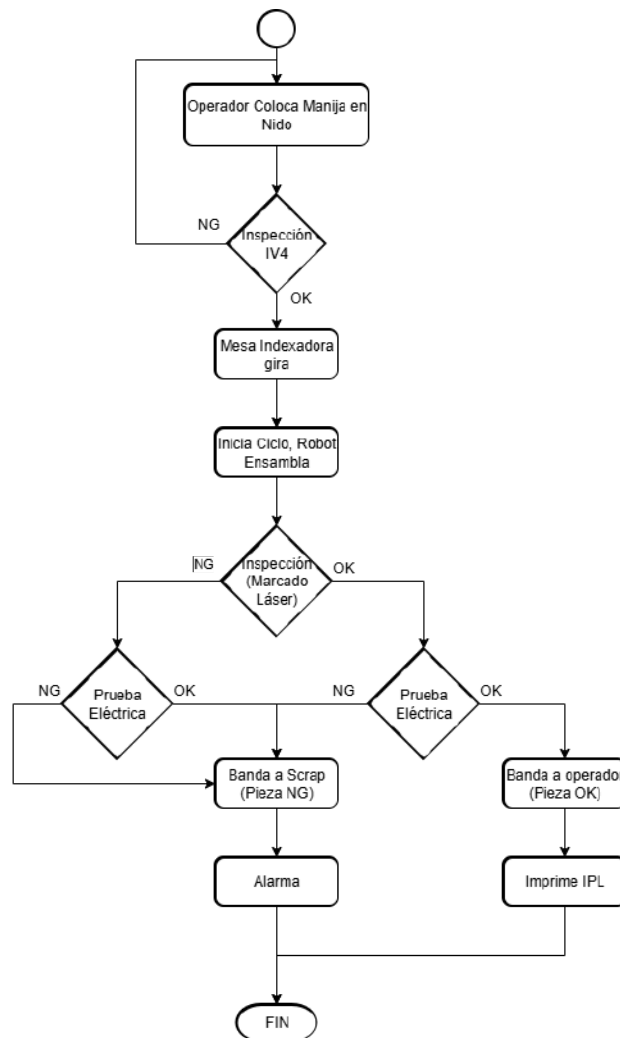


Figura 3.1: Diagrama de flujo de la secuencia de la estación de ensamble. Autoría propia, desarrollado en Mitchell Plastics Queretaro.

En paralelo a la secuencia descrita, se añadirán las funciones de IoT. Es decir, se instalará una Raspberry Pi 5 en la estación y se conectará por medio de un cable ethernet RJ45 al PLC Omron N1XP2.

3.1. Configuración Inicial de la Raspberry Pi 5

El primer paso que se siguió en la implementación del proyecto es la configuración inicial de la Raspberry Pi 5. En primer lugar, se debe de instalar el sistema operativo (SO) en el dispositivo. Para ello existe una herramienta llamada **Raspberry Pi 5 Imager**. La Figura 3.2 muestra la aplicación, como se puede ver es un paso a paso de instalación de software. En este caso, se utilizó el sistema operativo Debian Linux. Es requerido usar una memoria MicroSD en la Raspberry Pi 5 para que se almacene el sistema operativo y el resto de programas que se vieron en el Marco Teórico. Se instala en la MicroSD desde una laptop y luego se pasa la MicroSD (en este caso se usó una de 64 GB) a la Raspberry Pi 5 para que arranque.



Figura 3.2: Interfaz de Raspberry Pi Imager. Extraído de: <https://www.raspberrypi.com/news/a-new-raspberry-pi-imager/>

Dentro de esa aplicación se definen la contraseña y la dirección IP de la Raspberry Pi 5. Sin embargo, la versión de Debian que se utilizó para el proyecto causaba problemas entre la conexión WiFi al internet empresarial y la conexión por ethernet al Hub de la estación. Por ello fue necesario ejecutar los comandos de la Figura 3.5. El problema radica en cómo se gestionan las conexiones en la Raspberry, ya que hay un programa llamado NetworkManager. Se configura la dirección IP con el patrón 192.168.250.xx para que forme parte de la red LAN gestionandola manualmente. Eso se debe hacer para que Network Manager no pretenda establecer una conexión de internet a través de la conexión ethernet. Si se dejara el programa original, este provocaría que la conexión cayera cada pocos segundos al no detectar internet.

```
nmcli connection modify eth0 ipv4.method manual
nmcli connection modify eth0 ipv4.addresses 192.168.250.20/24
nmcli connection modify eth0 ipv4.gateway 192.168.250.20/24
nmcli connection modify eth0 ipv4.dns 192.168.250.20/24
nmcli connection modify eth0 ipv4.never default yes
```

```
nmcli connection modify eth0 connection.autoconnect
```

Figura 3.1: Comandos para gestionar la red ethernet sin Network Manager. Autoria propia, desarrollado en Mitchell Plastics Queretaro

3.2. Instalación de Programas

3.2.1. Mosquitto Broker y Node-RED

Una vez que se tiene el sistema operativo, se puede abrir una terminal de linux y comenzar a instalar los programas necesarios para el proyecto. El primero, es el broker **mosquitto**. En la siguiente Figura 3.2 se pueden apreciar los comandos necesarios para instalar adecuadamente el broker. El último comando instala Node-RED, este programa se instala a partir de un repositorio de GitHub por lo que la URL puede cambiar dependiendo del sistema operativo que se maneje. Cabe mencionar que los primeros dos comandos sirven para actualizar el sistema de linux por lo que es necesario ejecutarlo justo antes de instalar los programas para que se instalen versiones compatibles. La línea con `systemctl` sirve para verificar que el programa de mosquitto esté funcionando correctamente, de ser así se mostrará una palabra en color verde que dice “Active”. El mismo comando pero con node-red en lugar de mosquitto nos dará el status de node-red. Mientras que Mosquitto se activa automáticamente cada que se enciende la Raspberry, Node-RED incluye un archivo que debe habilitarse manualmente con la última línea de la Figura 3.2.

```
sudo apt update
sudo apt upgrade
sudo apt install mosquitto mosquitto clients
sudo systemctl status mosquitto
bash <(curl -sL https://github.com/node-red/linux-installers/releases/latest/download/update-nodejs-and-nodered-deb)
sudo systemctl enable nodered.service
```

Figura 3.2: Comandos para instalar Node-RED y Mosquitto Broker. Autoria propia, desarrollado en Mitchell Plastics Queretaro.

Tanto en estos programas como en los siguientes se hicieron las configuraciones pensando en que la activación sea totalmente automática, y en caso de que haya alguna falla, o se apague el sistema de manera imprevista, lo único que haya que hacer sea reconectar el dispositivo.

Para que la comunicación por MQTT entre el PLC y la Raspberry no se rompa, es necesario configurar adecuadamente el archivo de configuración de Mosquitto accediendo a él mediante el comando de la Figura 3.3 y colocar al

final del archivo las líneas que se ven en la misma figura. La que dice **listener 1883 0.0.0.0** le indica a MQTT que debe escuchar por medio del puerto 1883 a cualquier dirección IP que envíe mensajes ahí. Por otro lado, la que indica **allow_anonymous true** permite conexiones anónimas, aunque en realidad, las conexiones desarrolladas desde el PLC sí tienen un identificador.

```
sudo nano /etc/mosquitto/mosquitto.conf  
listener 1883 0.0.0.0  
allow_anonymous true
```

Figura 3.3: Comandos para habilitar conexiones desde otros dispositivos para MQTT, y crear un primer usuario. Autoría propia, desarrollado en Mitchell Plastics Queretaro.

3.2.2. Instalación de PostgreSQL

Para instalar PostgreSQL hay que ejecutar la Figura 3.4. La primera línea es para la instalación del software, la segunda es para acceder al usuario de administración postgres, la tercera es para entrar al propio programa de PostgreSQL y la última es para crear un usuario ajeno a postgres con el que se pueda comenzar a gestionar la base de datos.

```
sudo apt install postgresql  
sudo su postgres  
psql  
createuser (nombre) -P --interactive
```

Figura 3.4: Comandos para instalar PostgreSQL, y crear un primer usuario. Autoría propia, desarrollado en Mitchell Plastics Queretaro.

Por defecto, PostgreSQL no acepta cualquier conexión, es decir, en un principio PostgreSQL no acepta conectarse por medio de la dirección IP que se configuró anteriormente, solamente la IP local del dispositivo. Para ello se debe abrir un editor de texto en el archivo de configuración `pg_hba.conf`.

Se debe abrir la terminal del sistema operativo y escribir las líneas de la Figura 3.5, la primer línea permite moverse al directorio donde se encuentra el programa, luego “`sudo nano`” es el comando que abrirá el editor de texto de linux para el archivo al que se direcciona, como se puede ver, se entra a la carpeta de la versión de PostgreSQL, en este caso 17, luego a la carpeta main y finalmente al archivo de configuración.

```
cd /etc/postgresql  
sudo nano 17/main/pg_hba.conf
```

Figura 3.5: Comandos para editar el archivo de configuración de PostgreSQL. Autoría propia, desarrollado en Mitchell Plastics Queretaro.

Una vez abierto el archivo en el editor, se debe buscar la sección donde se definen las conexiones locales IPv4. En ella aparece la dirección local 127.0.0.1 que es la dirección IP local por defecto de todos los dispositivos. Se debe agregar la dirección definida anteriormente, en este caso 192.168.250.20 copiando el resto de datos como se aprecia en la Figura 3.3

```

# TYPE  DATABASE  USER  ADDRESS  METHOD
# "local" is for Unix domain socket connections only
local   all             all                                peer
# IPv4 local connections:
host    all             all             127.0.0.1/32      scram-sha-256
host    all             all             192.168.250.0/24  scram-sha-256
host    all             all             192.168.250.20/24 scram-sha-256
# IPv6 local connections:
host    all             all             ::1/128           scram-sha-256
# Allow replication connections from localhost, by a user with the
# replication privilege.
local   replication     all                                peer
host    replication     all             127.0.0.1/32      scram-sha-256
host    replication     all             ::1/128           scram-sha-256

```

Figura 3.3: Cambios en el archivo de configuración pg_hba.conf. Autoría propia, desarrollado en Mitchell Plastics Queretaro.

Adicionalmente se debe de abrir el archivo de configuración “postgresql.conf” mediante el comando de la Figura 3.6. Este archivo sirve para determinar desde qué direcciones IP es posible acceder a las bases de datos del dispositivo.

```

sudo nano /main/postgresql.conf.

```

Figura 3.6: Comando para editar el archivo de configuración de PostgreSQL. Autoría propia, desarrollado en Mitchell Plastics Queretaro.

Aquí, se debe buscar la línea del archivo que diga listen_addresses y colocar la direcciones IP adecuadas, o bien, un asterisco para indicar que la dirección IP sea accesible para todas las demás direcciones (es decir, que tengan acceso remoto).

3.2.3. Instalación de Grafana

El siguiente software a instalar es Grafana, la herramienta que servirá para generar los gráficos desde la base de datos. Al igual que con los anteriores programas, se deben ejecutar varios comandos que son visibles en la Figura 3.7.

```

# - Connection Settings -

listen_addresses = '*'          # what IP address(es) to listen on;
                                # defaults to 'localhost'; use '*' for all
                                # (change requires restart)
port = 5432                     # (change requires restart)
max_connections = 100          # (change requires restart)
reserved_connections = 0       # (change requires restart)
#superuser_reserved_connections = 3 # (change requires restart)
unix_socket_directories = '/var/run/postgresql' # comma-separated list of directories
                                # (change requires restart)
#unix_socket_group = ''        # (change requires restart)
#unix_socket_permissions = 0777 # begin with 0 to use octal notation
                                # (change requires restart)
#bonjour = off                 # advertise server via Bonjour
                                # (change requires restart)
#bonjour_name = ''             # defaults to the computer name
                                # (change requires restart)

```

Figura 3.4: Archivo de configuración para acceso remoto. Autoria propia, desarrollado en Mitchell Plastics Queretaro.

Este programa se descarga desde una URL en internet no sin antes crear un directorio llamado “keyrings” como se ve en la primera línea de la Figura 3.7. Luego, al igual que con Node-RED, se debe inicializar un servicio mediante los últimos dos comandos para que Grafana inicie automáticamente aún si se reinicia la Raspberry Pi 5.

```

sudo mkdir -p /etc/apt/keyrings

wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/
keyrings/grafana.gpg > /dev/null

echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable
main" | sudo tee /etc/apt/sources.list.d/grafana.list

sudo apt update

sudo apt install -y grafana

sudo systemctl enable grafana-server

sudo systemctl start grafana-server

```

Figura 3.7: Comandos para instalar y generar los archivos necesarios para Grafana. Autoria propia, desarrollado en Mitchell Plastics Queretaro.

Node-RED, Grafana y PostgreSQL son accesibles a través de cualquier dispositivo que esté dentro de la misma red que la Raspberry, si uno coloca la dirección IP de la esta en el navegador web podrá acceder a los tres programas añadiendo el puerto de acceso de cada uno a la URL dentro del buscador. Tanto Grafana como PostgreSQL cuentan con contraseñas por seguridad. Como Grafana se abre dentro de un navegador, la pantalla quedará invadida por las barras de navegación y las pestañas de las páginas web a menos que se añada un complemento al programa llamado Grafana Kiosk. Este programa inicia Grafana en pantalla completa, ello será útil para desplegar el panel en una pantalla instalada en la estación durante las horas de producción.

3.2.4. pgAdmin 4

Para instalar la interfaz gráfica que permitirá manejar las bases de datos creadas en PostgreSQL es necesario crear un entorno virtual de Python en donde correrá el programa. Una vez creado en él se añadirán seis carpetas: Una llamada “var” y dentro de esta se crearán las carpetas “lib” y “log”, y dentro de cada una de ellas se creará una carpeta “pgadmin”. A continuación, se activa el entorno virtual ejecutando un archivo bin llamado “activate” y se ejecuta, ya con el entorno operando, la línea “pip install pgadmin4” que iniciará la descarga del programa en sí. Todo esto se muestra línea a línea en la Figura 3.8

```
python3 -m venv pgadmin4
cd ~/pgadmin4
mkdir var
mkdir var/storage
mkdir var/lib
mkdir var/log
mkdir var/lib/pgadmin
mkdir var/log/pgadmin
source home/mitchell/pgadmin4/bin/activate
pip install pgadmin4
```

Figura 3.8: Creación de archivos e instalación de pgadmin4 en la terminal de linux. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

3.3. Variables del Proceso y Comunicación con MQTT

Si se revisa el diagrama de la Figura 3.1 se notará que hay dos variables que determinan el flujo del proceso y que, además, están correlacionadas con la calidad de las piezas ensambladas. La **Inspección del Marcado Láser** y la **Prueba Eléctrica**¹ que se vieron en el marco teórico. En este documento no se revisarán los criterios o programas desarrollados para determinar si la inspección de la cámara VS o la prueba eléctrica están en “OK” o “NG” ya que ello escapa al alcance del proyecto, pero téngase en cuenta que ambas variables se consideran binarias. Por otro lado es necesario crear un **identificador único** para cada pieza ensamblada con la intención de mejorar la trazabilidad de las manijas, y poder manejar los registros dentro de la base de datos de PostgreSQL. Recuérdese de la Tabla 2.1 de la sección de Elemento del Proceso en el Marco Teórico, que hay seis modelos de manijas diferentes; el ID o

¹La estación final de ensamblado tiene múltiples sensores para evaluar la calidad del producto final. Tanto el marcado láser como la adición del circuito interno de la manija se llevan a cabo en otras estaciones.

identificador único de cada ensamble será compuesto por la fecha en que terminó el ciclo del ensamble en formato YYYYMMDDHHmmss seguido de un guión bajo y el número de serie del modelo de manija.

Un aspecto considerado de gran importancia para la evaluación de los KPI de producción de la empresa es el tiempo de ciclo de cada ensamble (CT), por ello también se seleccionaron el momento de **inicio del ciclo**, el **momento de finalizado del ensamble** como variables a registrar. Por supuesto, también se consideró el **tiempo total del ciclo en segundos** como última variable del proceso. Por lo tanto, los registros que se llevarán a la base de datos quedarían como se ve en la Tabla.

Tabla 3.1: Ejemplo de registro que se llevará a la base de datos.

ID	vs_inspection	electrical_test	cycle_init	cycle_finish	cycle_total_time(sec)
20260422160345_42351A01	OK	NG	20260422 16:03:45	20260422 16:04:17	32

Antes de pasar a cómo se implementan los registros dentro de la base de datos con PostgreSQL, se revisará cómo se comunican el PLC Omron NX1P2 con la Rasoberry para ejecutar el registro.

El PLC, por medio de lenguaje estructurado es capaz de enviar una cadena de texto con formato JSON a través del protocolo MQTT. Para ello se utiliza una librería que permite crear un bloque varios bloques de función con las siguientes funciones.

- Establecer comunicación con la dirección IP del dispositivo.
- Suscribirse a un tópico.
- Enviar una variable como mensaje al tópico.

Como se ve en la Figura 3.6 concatenan las inspecciones, el identificador y los tiempos de ciclo en una cadena de texto común. En este caso, se establecerá la comunicación con el tópico “/AssemblyLH”. Para fines de este proyecto no se consideraron métodos de autenticación para no dificultar el despliegue a producción.

Se utilizó una librería especial para ejecutar los envíos de los mensajes JSON por MQTT, y establecer la conexión usando bloques de función. Como se muestra en la Figura 3.5, esa librería incluye seis bloques de función, pero en este caso, solo serán útiles cuatro. **MQTTClient** establece la conexión con el broker al poner en la entrada *Host* la dirección IP de la Rasoberry Pi 5, en *ServerPort* el puerto de comunicación, en este caso 1883, en *ClientID* un identificador exclusivo para la conexión ². La salida *Connected* al estar en 1 informará que la conexión se ha llevado con éxito. **MQTTPublish** es el bloque de función encargado de enviar el mensaje al tópico. En la entrada *topic* se coloca el tópico

²Es importante que cada conexión que se haga al broker tenga un identificador único. La experiencia desarrollando el proyecto demostró que si se coloca el mismo ID en varias conexiones, estas compiten entre ellas provocando errores

al que se envía el mensaje, “/AssemblyLH” en este caso; en la entrada *Message* se coloca el JSON que transportará la información a la Raspberry Pi 5 a través del bloque; *QoS* se pone en 1 para enviar un solo mensaje. La salida *Done* al ponerse en 1 indicará que se ha enviado el mensaje correctamente. **MQTTSubscribe** no se ha utilizado para este proyecto, pero es importante diferenciar este bloque de función del anterior; este funciona para “escuchar” los mensajes en el tópico, mientras que el anterior para enviarlos al tópico. **MQTTPing** se usa únicamente para monitorear si se encuentra activa la conexión.

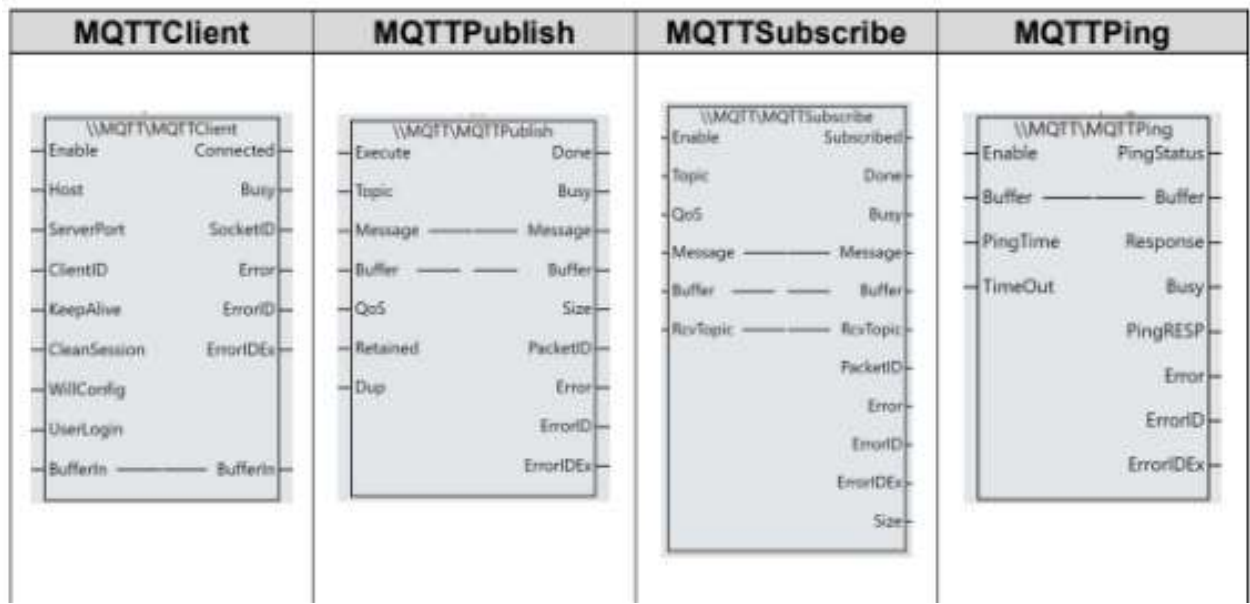


Figura 3.5: Bloques de función para establecer conexión con el broker mosquitto, publicar, enviar ping de conexión, y suscribirse a tópicos. Obtenida de [https://www.infopl.net/descargas/omron/controlador-nj-sysmac/omron-sysmac-studio-librer %C3 %ADa-mqtt-en](https://www.infopl.net/descargas/omron/controlador-nj-sysmac/omron-sysmac-studio-librer-%C3%ADa-mqtt-en)

```
1 MessageMQTT:=
2 ("Seguir la sintaxis de {\"clave\":\"valor\"}, recordar dobles comillas")
3 CONCAT( // Cada CONCAT adiciona un dato al JSON
4   CONCAT(
5     \"{id\":\", valuesMQTTJSON[0] //id de pieza
6   ),
7   CONCAT(
8     \", \"vs_inspection\":\", valuesMQTTJSON[1] //Inspección de marcado
9   ),
10  CONCAT(
11    \", \"electrical_test\":\", valuesMQTTJSON[2] //Prueba Electrica
12  ),
13  CONCAT(
14    \", \"cycle_init\":\", valuesMQTTJSON[3] //Inicio del ciclo
15  ),
16  CONCAT(
17    \", \"cycle_finish\":\", valuesMQTTJSON[4] //Final del Ciclo
18    \", \"cycle_total\":\", valuesMQTTJSON[5] //Tiempo total del ciclo
19  )
20  )
21 )
```

Figura 3.6: Construcción del JSON en Sysmac Studio (La obtención de las variables no corresponde a este proyecto y el código para procesarlas es de uso exclusivo para Mitchell Plastics). Autoría propia, desarrollado en Mitchell Plastics Querétaro.

Entonces al hacer el envío a la Raspberry a este tópico, cualquier programa podrá ver el mensaje con el JSON siempre y cuando cuente con un MQTT Broker. El programa Node-RED será la principal puerta de enlace al resto de programas para aprovechar los envíos de datos desde el PLC.

3.4. Procesamiento de los Registros en Node-RED

El flujo de la Figura 3.7 muestra la lógica para insertar los registros en la base de datos. Como se puede apreciar el flujo consta de tres nodos. El primero recibe todos los mensajes de MQTT en el tópico. El segundo genera el objeto que será enviado por el flujo al siguiente nodo. Realmente, este nodo solo traduce el contenido de msg.payload, es decir, la cadena de texto generada en Sysmac Studio a un objeto de JavaScript. Una vez que se tiene el objeto, este se manda a un nodo de PostgreSQL donde se configura la conexión con la base de datos en la dirección IP y el puerto de acceso dados a PostgreSQL y se hace una inserción con SQL llamando a las variables del objeto mediante el símbolo de dinero. Ese símbolo llama exclusivamente a aquellas variables almacenadas en el array msg.params.

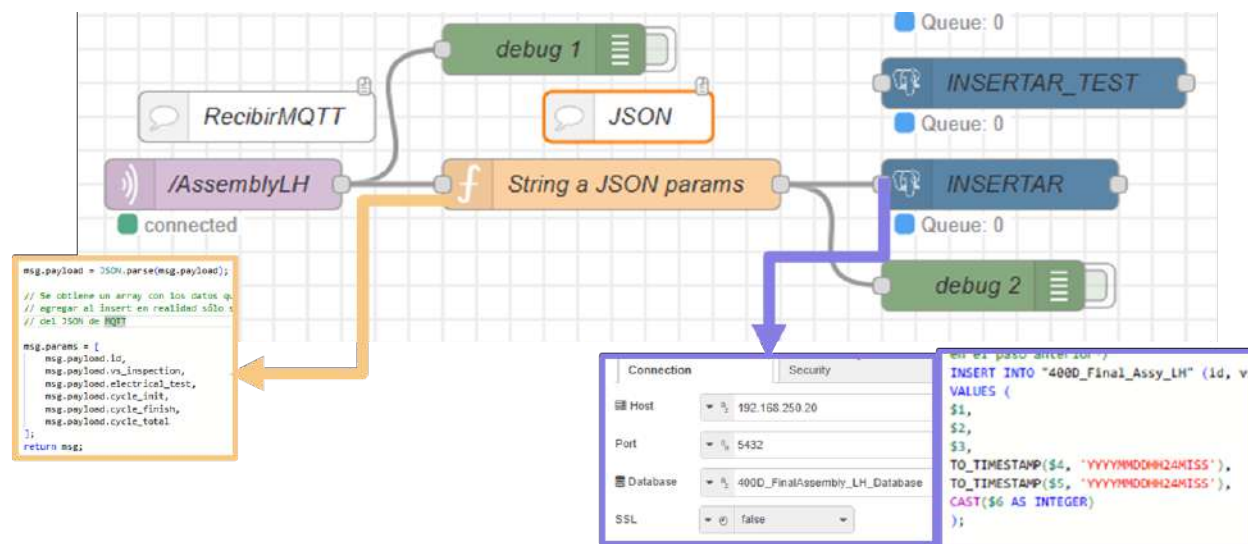


Figura 3.7: Flujo y contenido de los nodos en Node-RED. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

Desde luego, antes de hacer este flujo, PostgreSQL ya debe estar configurado y deben crearse los elementos de los que se hará consulta.

3.5. Creación de Base de Datos

En PostgreSQL se implementó una base de datos llamada 400D_FinalAssembly_LH_Database, dentro de ella se creó una con el nombre 400D_Final_Assy_LH con la estructura antes mencionada con seis columnas incluyendo un identificador único. La base de datos será consultada por múltiples programas para hacer reportes, cálculos, etc. Por

lo cual se crearon hasta 12 vistas en la base de datos para facilitar el manejo de los mismos. Se incluyen las piezas producidas por turno, día y las de la fecha actual, los conteos de inspecciones y las piezas producidas por hora. En la Figura 3.8 se muestran estos elementos en pgAdmin 4.

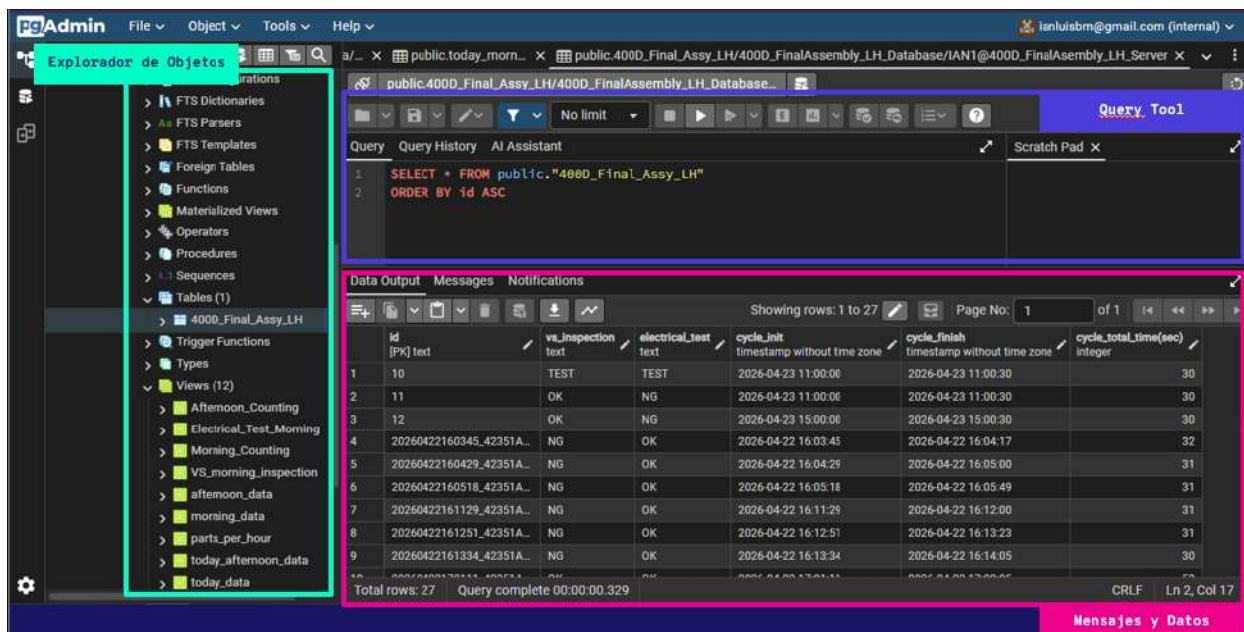


Figura 3.8: Base de datos en pgAdmin 4. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

3.6. Programación de Reportes de Producción por Turno en Node-RED

Se crearon dos flujos para enviar automáticamente un reporte de producción tanto a las 14 como las 22 horas. Cuando llega la hora el nodo inject del flujo inicia dos secuencias simultáneas; la primera consulta a la base de datos y calcula cuatro estadísticas, el porcentaje de piezas OK por prueba eléctrica, lo mismo pero con la inspección de marcado láser, la cantidad de piezas producidas, y el promedio del tiempo de ciclo. La otra rama consulta todos los registros del turno, ya sea desde las 6 hasta las 14 horas, o desde las 14 a las 22, y luego lo manda a un nodo CSV. Ese nodo transforma los datos del turno en un archivo csv. A ambas ramas se les asigna un topic dentro del objeto msg. Un nodo join une ambos resultados de las consultas en un solo objeto si y solo si ambos tópicos llegan al nodo. Se transmite el msg completo a un nodo Function que creará el formato en HTML para el email, y al final en un nodo configurado con las credenciales del servidor interno de la empresa se encargará de enviar el email. Todo ello se muestra en la Figura 3.9, recuérdese que los nodos verdes oscuros debug sirven para manejo de errores y monitorear el estado del flujo.

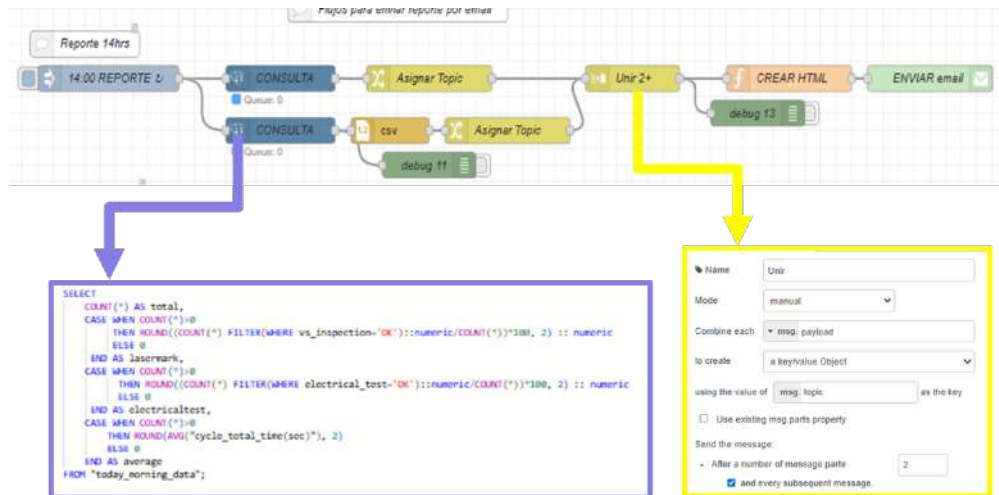


Figura 3.9: Flujo de envío de reportes de producción por turno en Node-RED. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

3.6.1. Formato de Email

En la Figura 3.9 se muestra cómo se programa el contenido del email. Del código, se identifican los siguientes elementos del objeto msg del flujo.

```

28 let today = new Date()
29 msg.topic="Reporte de Turno Matutino 4000 Final Assy";
30 msg.attachments=[{
31   filename: `datos_matutino_${today.toISOString().slice(0,10)}.csv`,
32   content: msg.payload.csv // El contenido CSV generado
33   | | |   }];
34
35 msg.payload=
36
37   <html>
38     <h1>Reporte de Turno</h1>
39     <p>
40       Estas son las estadísticas registradas durante la operación de la
41       estación de ensamblado final 4000 en el turno matutino de 6:00 a 14:00.
42       <ul>
43         <li><b>${msg.payload.calculos[0].total}</b> piezas producidas.
44         <li><b>${msg.payload.calculos[0].lasermark}</b> Marcado Laser OK.
45         <li><b>${msg.payload.calculos[0].electricaltest}</b> Prueba eléctrica OK.
46         <li><b>${msg.payload.calculos[0].average}</b> promedio de tiempo de ciclo.
47       </ul>
48       Revise el archivo adjunto para ver las piezas producidas durante el turno.
49       <br>
50     </p>
51   </html>
52
53 return msg;
  
```

Figura 3.10: Código para generar el formato del Email. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

Para los reportes de producción se hicieron unos cálculos convenientes para su análisis dentro de la gestión interna de la empresa. Por un lado, se hace un conteo del total de piezas producidas durante un turno. Eso se hace mediante el código, en este caso una consulta que aparece en la Figura 3.10. En este caso mediante el comando COUNT(*) que cuenta todas las filas dentro de la tabla a la que se hace referencia. Por otro lado, se calcula el porcentaje de piezas OK en la inspección de marcado láser. Además se calculan los porcentajes con la Ecuación 1 tomando las piezas OK (p_OK), dividir las entre el total de piezas (total) y multiplicarlas por 100.

$$\frac{p_OK}{total} * 100 \quad (1)$$

Lo anterior aplica tanto para las inspecciones de marcado láser como las pruebas eléctricas. Además, se agregó el cálculo del promedio de tiempo de ciclo en la producción. Para ello, se utilizó la Ecuación 2, que consiste en sumar todos los tiempos de ciclo de la producción de cada ensamble CT de cada registro n y dividirlos entre la cantidad de registros M.

$$\frac{\sum_{n=1}^M CT(n)}{total} \quad (2)$$

3.7. Desarrollo de Alertas del Sistema

Como parte del sistema de monitoreo de calidad de la estación de ensamblado hay dos escenarios que se desean monitorear en los reportes en caso de que sucedan. El primero, si hay tres piezas NG durante alguna hora determinada de la producción, o, en el segundo caso, si ocurren cinco ciclos más lentos de lo habitual, se toma en cuenta a un ciclo de 50 segundos o más un ciclo lento. Se programó el PLC para que este mandara las activaciones de las alertas a través de MQTT construyendo el JSON que se aprecia en la Figura 3.11.

```

1 MessageMQTT3:=
2 CONCAT( // Cada CONCAT adiciona un dato al JSON
3     CONCAT(
4         "["alerta":", INT_TO_STRING(alertaMQTTNo)
5     ),
6     CONCAT(
7         ",", "ciclo_final":", valuesMQTTJSON[5]
8     ),
9     CONCAT(
10        ",", "hora":", hmi_sClock,
11        "]"
12    )
13 );

```

Figura 3.11: Construcción del JSON para enviar señales de alerta. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

En este caso se mandará el JSON a un tópico de MQTT diferente a los de producción. Se trata del tópico /AlertEmail. Como se observa en el flujo de la Figura una vez más se obtiene una consulta a PostgreSQL de cuáles han sido las piezas NG durante la hora actual y en la rama paralela a esta, se da formato de objeto al JSON y ambas ramas confluyen en un nodo join que une ambos resultados o caminos del flujo. Para identificar las alertas en el software Sysmac se programó que se enviara una variable “alerta” que puede tomar el valor de 1 si la alerta es de ciclos lentos o 2 si la alerta es de calidad (ensambles NG). Un nodo switch lee esta variable desde el objeto generado a partir del JSON y envía el flujo a una de dos ramas que son idénticas en cuanto a los nodos que usan, pero cada una tiene un formato de email diferente que se mostrarán en la sección de resultados.

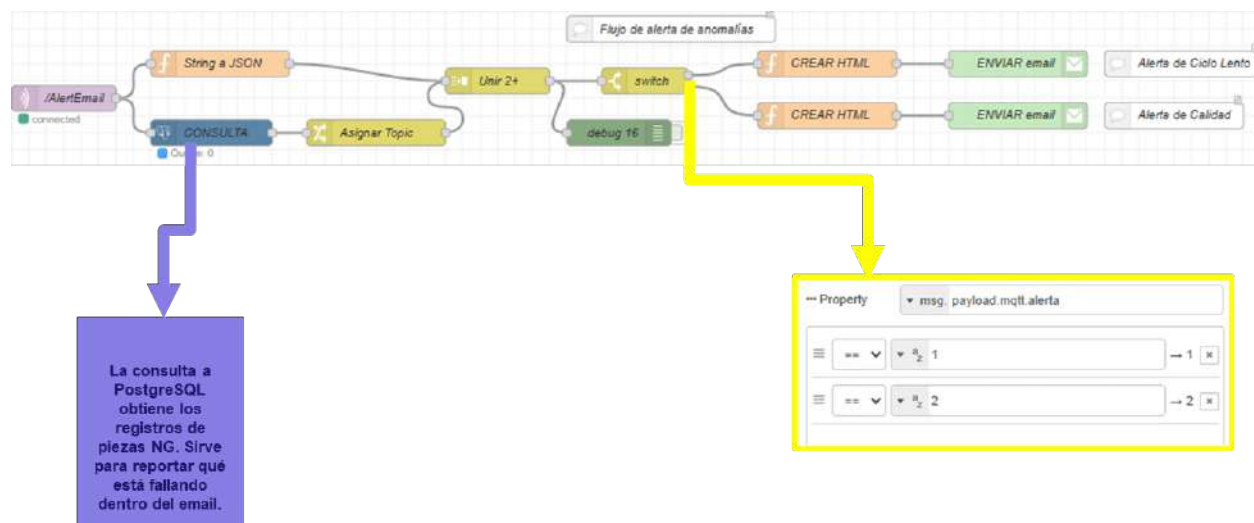


Figura 3.12: Flujo de manejo de alertas. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

Para realizar la configuración correcta de los envíos de correos usando el servidor privado de la empresa se encontró que es necesario utilizar una librería denominada nodemailer en Node-RED. En la Figura ?? se muestra cómo se configura el envío haciendo uso del mensaje HTML que se mostró anteriormente por medio de JavaScript. Primero se configura un objeto **transporter**, *pool* permite almacenar varios mensajes en caso de que se que aún no haya terminado el proceso de envío algún mensaje anterior³. *host* es el servidor SMTP por el cual se enviará el correo, ese fue configurado por la empresa. *Port* es el puerto con un cifrado específico para enviar el mensaje, en este caso se usa el 25 que no tiene cifrado. *connectionTimeout*, *greetingTimeout* y *socketTimeout* permiten configurar los tiempos de espera para recibir saludos y mensajes de confirmación en node red.

El objeto **mailOptions** permite configurar las opciones de envío del correo dentro del servidor. Se configura el atributo *from* con el emisor del correo, *to* con el destinatario del mismo. *Subject* contiene el asunto del correo, y *html* especifica el contenido en HTML que se enviará el mensaje.

³Durante el desarrollo del proyecto se notó que el servidor tarda mucho tiempo en responder (alrededor de minuto y medio)

Las últimas seis líneas de código son para ejecutar el envío en sí. Se aplica la palabra clave *await* para señalar que se debe ejecutar uno por uno el proceso de envío cada vez que llegue un nuevo correo para evitar saturar el servidor, o que este confunda el proceso con un ataque cibernético. Se usa el método *.sendMail* en el transportador configurado para ejecutar el envío, y se pone como argumento del método las opciones colocadas en *mailOptions*

```
1  const nodemailer = require('nodemailer');
2
3  // 1. Configurar el transportador (servidor SMTP)
4  const transporter = nodemailer.createTransport({
5      pool: true
6      host: 'host de la empresa',
7      port: 25,           // se configuro un puerto seguro con IT
8      secure: false
9      connectionTimeout: 1000, // Tiempo para establecerTCP
10     greetingTimeout: 1000,   // Tiempo para esperar el saludo del servidor SMTP
11     socketTimeout: 1000,     // Tiempo de inactividad en el socket abierto
12 });
13
14 // 2. Definir los datos del correo
15 const mailOptions = {
16     from: 'mpq400d.tracking@mitchellplastics.com',
17     to: 'ian.barcenas@mitchellplastics.com',
18     subject: 'Reporte de Turno 400D',
19     html: msg.payload
20 };
21
22 // 3. Enviar el correo
23 await transporter.sendMail(mailOptions, (error, info) => {
24     if (error) {
25         return console.log('Error al enviar:', error.message);
26     }
27     console.log(' Correo  enviado con exito!');
28 });
```

Figura 3.9: Configuración de nodemailer para enviar correos electrónicos, Autoría propia

3.8. Desarrollo de Tablero en Grafana

Paralelamente a la operación de Node-RED, se ejecuta Grafana para mostrar un tablero de visualizaciones en una pantalla por medio de HDMI. Se ha elegido colocar dos tableros en una playlist que hará que estén intercambiándose en la pantalla cada 30 segundos.

El primer tablero es el de **producción hora por hora**. Este es una Business table de Grafana descargada como un plugin de la comunidad. La tabla se diseñó para que mostrará en rojo si la producción aún no llega a los valores esperados de producción u órdenes de trabajo (se estima que serán 120 por hora), en amarillo si está a punto de alcanzar el objetivo y verde si el objetivo de producción ha sido alcanzado. Para desarrollarlo se siguieron los siguientes pasos:

1. Se estableció el layout de la pantalla para tener tres celdas distribuidas por la pantalla.
2. La primer celda muestra un texto en HTML que indica el título del tablero.
3. La segunda celda al lado del título muestra la hora y fecha actual. Se trata de una visualización tipo Clock.
4. La tercera es la propia tabla. Primero es necesario insertarla en la celda
5. Se programa una consulta directa a la base de datos para extraer la vista de la producción hora por hora.
6. Con HTML se ajusta el tamaño del texto de la tabla.
7. Se configura la tabla con las opciones por defecto al hacer click en editar el panel.

Se creó otro tablero que será el que recopilará las inspecciones que se han hecho por los sensores. Constará de dos gráficas de pastel y cuatro gráficas de barras debajo de las de pastel. Adicionalmente se agregó el total de piezas producidas. La secuencia para agregar este tablero es parecida al anterior:

1. Se divide la pantalla en 12 celdas.
2. Las primeras dos son de título, y hora y fecha.
3. Dos celdas para las dos gráficas de pastel que corresponden a cada turno de producción (mañana y tarde).
4. Debajo de las gráficas de pastel, las gráficas de barras muestran las tendencias en los valores de las inspecciones.
5. Todos los elementos salvo los de texto de este tablero requieren hacer consultas a la base de datos.

Capítulo 4

Resultados

4.1. Programas Realizados

Los primeros programas que se desarrollaron fueron los de Sysmac Studio que no se pueden colocar en este informe por razones de confidencialidad de la empresa, pero se puede proveer del esquema del flujo de alertas el cual es el que se muestra en la Figura 4.1.

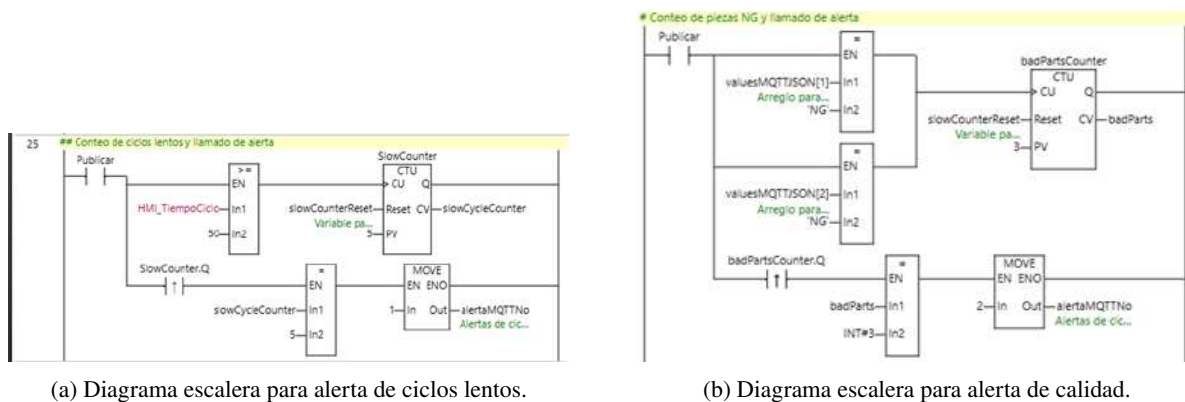


Figura 4.1: Diagramas de escalera para alertas del proceso. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

Luego, como se muestra en la Figura 4.2 se hizo un programa para Node-RED. Los flujos están distinguidos por cuadros de colores; el cian es el de inserción de datos en la base de datos. El morado es para los envíos de emails automáticos, y el guinda es para las alertas.

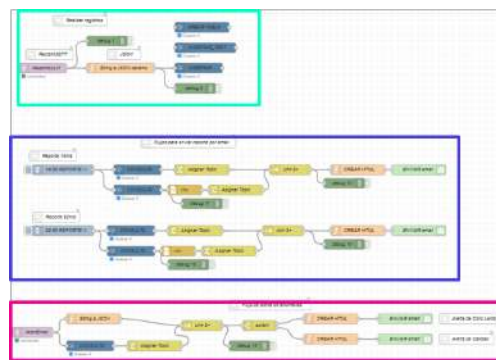
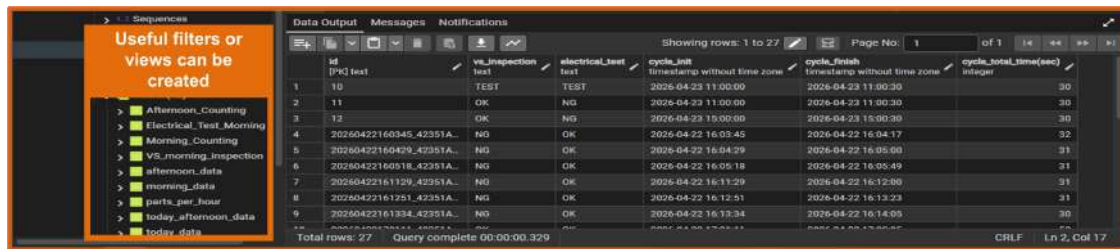


Figura 4.2: Flujos desarrollados en Node-RED. Autoría propia, desarrollado en Mitchell Plastics Querétaro

Los programas fueron configurados para ejecutarse siempre que la raspberry Pi 5 esté encendida, es decir se colocaron dentro del arranque del dispositivo. Inherentemente a los flujos presentados están los envíos de emails automáticos para reportar estadísticas

4.2. Base de Datos

Se implementó la base de datos, como se mostró anteriormente, hace los registros tomando en cuenta las principales métricas para producción. En la Figura 4.3 se muestran los registros que se realizan en producción. Se comprobó que los registros se realizan sincronizadamente con la adquisición de los datos de la estación. Adicionalmente a la tabla donde se hacen los registros, también se agregaron vistas convenientes que incluyen: registros del día, conteo de piezas por hora, conteo de piezas por turno, conteo de piezas OK en prueba eléctrica, conteo de piezas NG por inspección de marcado láser.



ID	[PK] test	vl inspection test	electrical_test test	cycle_start timestamp without time zone	cycle_end timestamp without time zone	cycle_total_time(sec) integer
1	10	TEST	TEST	2026-04-23 11:00:00	2026-04-23 11:00:30	30
2	11	OK	NG	2026-04-23 11:00:00	2026-04-23 11:00:30	30
3	12	OK	NG	2026-04-23 15:00:00	2026-04-23 15:00:30	30
4	20260422160345_42351A	NG	OK	2026-04-22 16:03:45	2026-04-22 16:04:17	32
5	20260422160429_42351A	NG	OK	2026-04-22 16:04:29	2026-04-22 16:05:00	31
6	20260422160518_42351A	NG	OK	2026-04-22 16:05:18	2026-04-22 16:05:49	31
7	20260422161129_42351A	NG	OK	2026-04-22 16:11:29	2026-04-22 16:12:00	31
8	20260422161251_42351A	NG	OK	2026-04-22 16:12:51	2026-04-22 16:13:23	31
9	20260422161334_42351A	NG	OK	2026-04-22 16:13:34	2026-04-22 16:14:05	30

Total rows: 27 Query complete 00:00:00.329 CRLF Ln 2, Col 17

Figura 4.3: Base de datos operativa. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

4.3. Tableros en Grafana.

Como se mencionó antes se crearon dos tableros en una lista de reproducción. En la Figura 4.4 se aprecia la implementación en planta. Los tableros van rotando cada treinta segundos y ambos actualizan sus datos (directamente conectados con la base de datos) de manera totalmente automática. La instalación física del sistema contempló lo siguiente:

- Instalar un cable de ethernet RJ45.
- Instalación de la Raspberry Pi 5 en un lugar cercano a la pantalla.
- Conexión de HDMI con la pantalla para visualizar Tablero
- La instalación de la pantalla fue hecha por trabajadores del área de mantenimiento.

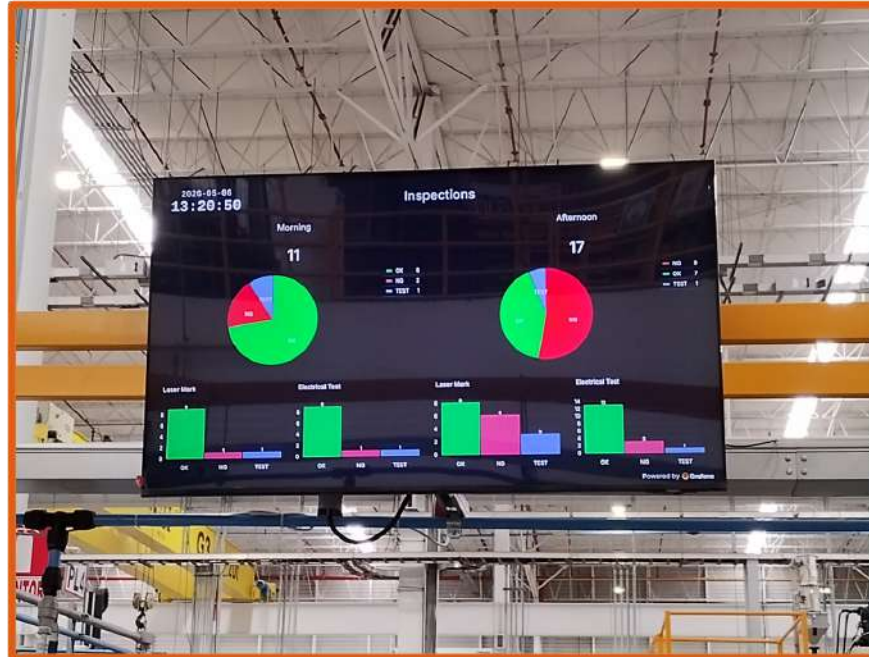


Figura 4.4: [Pantalla en la estación 400D con tableros]Pantalla en la estación 400D con tableros. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

También en la Figura 4.5 se aprecia como es la tabla de producción hora por hora.



Figura 4.5: [Pantalla en la estación 400D con tableros]Tablero con tabla de producción hora por hora. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

4.4. Emails Automáticos

En la Figura 4.6 está el email de reporte de turno. En ese email se puede observar que contiene tanto el texto como cálculos mencionados en la parte de desarrollo. Como se puede ver, el texto está formateado con ayuda de código en

HTML. Las estadísticas se presentan en formato de lista y el archivo CSV generado se agrega al correo. En este caso se deben enviar los correos no a través de los servidores de email sino a un servidor de la empresa que gestiona correos en outlook.



Figura 4.6: Email reporte de turno. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

En la Figura 4.7 se encuentra el formato de la alerta activada por la presencia de demasiadas piezas NG en el proceso, en este caso se envía información sobre la fecha en que se detectó la tercera pieza NG consecutiva, las piezas NG cuyo problema fue la prueba eléctrica y las piezas NG que fallaron la inspección de marcado láser. Se hace un conteo de esas piezas. Debe destacarse que esta alerta se activa solamente bajo la condición de que se detecten tres piezas NG en la misma hora. La alerta solo se envía en una ocasión en una hora determinada y la bandera de activación se restablece con el cambio de hora.



Figura 4.7: Email de alerta de calidad. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

Ahora, en la Figura 4.8, se encuentra la alerta de ciclos lentos. Se puede apreciar que al igual que con la alerta de calidad, se registra la fecha en que se detectó el problema y, además, se envía la duración del último ciclo lento o el que detonó la ejecución de la alerta. Tiene un funcionamiento similar a las alertas de calidad, solo se envía una vez por

hora y bastan cinco ciclos mayores a 50 segundos en una hora para desencadenar la alerta.



Figura 4.8: Email de alerta de ciclos lentos. Autoría propia, desarrollado en Mitchell Plastics Querétaro.

Los emails pueden mejorarse en un futuro y sobretodo se espera mayor conectividad con los software que se utilizan en planta como el ERP de Rockwell Automation PLEX. La estructura actual del proyecto ha demostrado utilidad para aplicarse a proyectos nuevos en la empresa.

Capítulo 5

Conclusiones

Con base en los resultados obtenidos se concluye que el sistema tiene un comportamiento funcional, y que ha cumplido con los diversos criterios de éxito que se han ido estableciendo con la elaboración del proyecto. Los registros se hacen de manera íntegra y no ha habido bugs en los programas. Los diferentes objetivos específicos que se establecieron fueron cumplidos dando a lugar los tableros de monitoreo de producción de la estación.

Como primer punto, debe decirse que este proyecto ha abierto muchas posibilidades de desarrollo en la empresa. Dado que se utilizaron elementos de código abierto, es decir, sin costo adicional y con una comunidad de desarrollo enorme, el proyecto goza de una enorme escalabilidad a corto plazo. En la empresa, se han hecho solicitudes y planes para agregar funcionalidades nuevas, como manejo de API's desde la estación, incorporación de visión artificial para análisis de ergonomía, actualización de equipo viejo de la planta, etc. Los gerentes de alto nivel de la planta han mostrado su interés en que esas aplicaciones se desarrollen.

Los costos son bajos para este proyecto, ya que el mayor costo fue la adquisición de la Raspberry Pi 5 que está alrededor de los 350 dólares. Considerando más elementos necesarios, como la pantalla en la que se presentan los tableros, se llegó a un costo de 1000 dólares. Además el tiempo de desarrollo es bastante rápido, si bien, se requiere mucho conocimiento técnico para realizar lo que en este proyecto se ha logrado, la información disponible en internet es mucha. Ello es una ventaja considerable frente a otros dispositivos cuyo software requiere pago de licencias y su desarrollo está limitado a diferencia de la Raspberry Pi 5 donde la flexibilidad en el desarrollo permite llegar a las mismas soluciones con diferentes caminos.

Cualquiera que sea el propósito de modificaciones futuras, o la lectura de este reporte para nuevos proyectos, se recomienda siempre seguir estándares para generar códigos que sean fácilmente mantenibles. El mayor reto a la hora de desarrollar el proyecto fue lograr una configuración eficiente en la raspberry pi 5 para cubrir todas las necesidades de la estación. Documentar todo es esencial para no perder esa configuración que se puede emplear en muchos más proyectos.

Otra desventaja que se ha encontrado es la multidisciplinariedad que demanda hacer un proyecto como este. Un aspecto donde el uso de dispositivos más caros o con pagos de licencia es mejor, es el de desarrollo todo en uno,

ya que en el proyecto se implementó Grafana, Node-RED, donde además se usó JavaScript y HTML, PostgreSQL, y la terminal de Debian. Implementar otras aplicaciones, según investigaciones que se han hecho posteriormente al término del proyecto requieren aún más herramientas. No obstante, este punto depende esencialmente de los alcances del proyecto.

Aunque la empresa proporcionó su apoyo constante en el desarrollo de este proyecto, se debe mencionar que la escalabilidad del mismo, o incluso para la implementación de algunas de las funciones que se presentaron en este reporte, dependen de la colaboración con otros departamentos especialmente de IT para manejar las credenciales de acceso y conectividad Wifi de la Raspberry Pi 5. Esto resultó en ocasiones un obstáculo ya que se ha requerido de administración por parte de altos cargos en la empresa para implementar el wifi en el dispositivo.

Capítulo 6

Competencias Desarrolladas

1. **Programación y conexión de PLC:** Durante el proyecto se realizó la conexión física de PLC Omron modelo NX1P2 además de su programación en lenguaje escalera y estructurado.
2. **Redes Industriales:** Fue necesario establecer comunicación mediante MQTT y Ethernet/IP en la ejecución del proyecto e incluso la elaboración de cables ethernet.
3. **Raspberry Pi 5.** Se usó como principal herramienta la Raspberry Pi 5 incluyendo su instalación, armado, comunicación, configuración, manejo de errores, instalación de programas, manejo de archivos etc. Concretamente se empleó Debian como distribución de linux para el proyecto.
4. **Node-RED:** La lógica de proceso se encuentra principalmente encapsulada en Node-RED, programa que se aprendió a usar sobre la marcha.
5. **PostgreSQL y pgAdmin4:** Se desarrolló la base de datos de la aplicación con ayuda de PostgreSQL incluyendo con ello el desarrollo de comandos en SQL.
6. **Manejo de Cámaras VS e IV4 de Keyence:** Estas cámaras si bien no forman parte como tal del proyecto sí que fue necesario entenderlas y en ocasiones programarlas para mejorar el desempeño del proyecto.
7. **JavaScript:** Se hicieron varios scripts en este lenguaje de programación, para ejecutar tareas muy específicas como cálculos, manejo de objetos o hash tables, ciclos, etc.
8. **HTML:** Se usó en la implementación para formatos de emails principalmente.
9. **Grafana:** Se empleó este software para desarrollar las visualizaciones resultantes del resto de programas.
10. **Robot Fanuc:** Al igual que con las cámaras inteligentes, fue menester entender el funcionamiento del robot, y en ocasiones controlarlo para manejo de errores, mejoras, y desarrollo de pruebas.
11. **HMI Omron y Keyence:** Como parte de la integración del nuevo sistema fue necesario hacer cambios en HMI's de la marca Omron y de Keyence. Si bien no formó parte del proyecto principal, sí que se recibió capacitación para operar y programar estas HMI.

Bibliografía

- Aceitón, J. E. M. (2020). El IoT-PLC: Una Nueva Generación De Controladores Lógicos Programables Para La Industria 4.0. *Pontificia Universidad Catolica de Chile ProQuest Dissertations Theses*.
- KEYENCE. (2026a). *Sistema de Visión con IA integrada*. <https://www.keyence.com.mx/products/vision/vision-sys/vs/>
- Corporation, F. A. (2016). *HandlingTool Operations & Programming Student Manual*.
- TecnoDigital. (2025). *PLC: qué es, cómo funciona y para qué sirve en la automatización industrial*. <https://informatecdigital.com/que-es-un-plc/>
- KEYENCE. (2026b). *Industria 4.0 en México: Qué es y Cómo Implementarla en 2026*. <https://www.keyence.com.mx/products/vision/vision-sys/vs/>
- raspberrypi.com. (2026). *Raspberry Pi 5*. <https://www.raspberrypi.com/products/raspberry-pi-5/>
- Pradyumna, G., Omkar, B., & Sagar, B. (2018). Introduction to IOT. *International Advanced Research Journal in Science, Engineering and Technology*.
- Automation, R. (2000). EtherNet/IP descripción general del sistema. *Rockwell International Corporation*.
- Lüke, J. P. (2019). Guía sobre direccionamiento IP, subredes y enrutamiento. *riull.ull.es*.
- Soni, D., & Makwana, A. (2017). A survey on MQTT: A Protocol of Internet of Things (IoT). *International conference on telecommunication, power analysis and computing techniques*.
- Logicbus. (2024). *Mosquitto MQTT. Presentación y guía básica de uso*. <https://www.logicbus.com.mx/blog/mosquitto-mqtt-presentacion-y-guia-basica-de-uso/>
- Bidgoli, H. (2006). *Handbook of Information Security Threats, Vulnerabilities, Prevention, Detection, and Management*. John Wiley & Sons.
- Marrs, T. (2017). *JSON at Work: Practical Data Integration for the Web*. O'Reilly Media.
- Salomón, R. E. R. (2012). *El gran libro de Debian GNU/Linux*. Marcombo.
- Shotts, W. (2019). *The Linux Command Line*. No Starch Press.
- Corporation, O. (2026). *Sysmac Studio*. <https://industrial.omron.es/es/products/sysmac-studio#features>
- Hussein, F. A., Yaseen, M. T., & Mustafa, M. O. (2025). The Future of Intelligent Industrial Systems: PLC, Node-RED, and IoT/IIoT. *AUIQ Technical Engineering Science*.
- O'Leary, N., & Conway-Jones, D. (2026). *Low-code programming for event-driven applications*. <https://nodered.org/>
- Sveakis, L. L., van Putten, M., & Percival, R. (2021). *JavaScript from Beginner to Professional*. Packt Publishing.
- Casado-Vara, R. (2019). Introducción a HTML. *Ediciones Universidad de Salamanca*.
- Group, T. P. G. D. (2026). *PostgreSQL 18.4 Documentation*. PostgreSQL Global Development Group.

Salituro, E. (2023). *Learn Grafana 10.x: A beginner's guide to practical data analytics, interactive dashboards, and observability*. Packt Publishing Ltd.